

SOLIDWORKS API (NET VBA C#) — Examples (22/50): Get_All_Sheet_Metal_Feature_Data_Example- Get_and_Set_Seed_Components_Example_VB

SOLIDWORKS API Documentation (converted from CHM)

Get All Sheet Metal Feature Data Example (VB.NET)

This example shows how to get all of the sheet metal part's feature data.

```

'-----
' Preconditions:
' 1. Open a sheet metal part.
' 2. Open the Immediate window.
'-----
' Postconditions: Inspect the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swFeat As Feature
    Dim swSubFeat As Feature
    Dim bRet As Boolean
    Dim lRet As Integer
    Dim gaugeTableFile As String

    Sub Process_CustomBendAllowance _
    ( _
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swCustBend As CustomBendAllowance _
    )

        Debug.Print("      BendAllowance      = " & swCustBend.BendAllowance * 1000.0# & " mm")
        Debug.Print("      BendDeduction      = " & swCustBend.BendDeduction * 1000.0# & " mm")
        Debug.Print("      BendTableFile      = " & swCustBend.BendTableFile)
        Debug.Print("      KFactor             = " & swCustBend.KFactor)
        Debug.Print("      Type                = " & swCustBend.Type)

    End Sub

    Sub Process_SMBaseFlange _
    ( _
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
    )

        Debug.Print("  +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

        Dim swBaseFlange As BaseFlangeFeatureData
        swBaseFlange = swFeat.GetDefinition

        Debug.Print("      BendRadius = " & swBaseFlange.BendRadius * 1000.0# & " mm")

    End Sub

    Sub Process_SheetMetal _
    ( _
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
    )

        Debug.Print("  +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

        Dim swSheetMetal As SheetMetalFeatureData
        swSheetMetal = swFeat.GetDefinition

        Dim swCustBend As CustomBendAllowance
        swCustBend = swSheetMetal.GetCustomBendAllowance
        Debug.Print("      BendRadius = " & swSheetMetal.BendRadius * 1000.0# & " mm")

        Process_CustomBendAllowance(swApp, swModel, swCustBend)

        lRet = swSheetMetal.GetUseGaugeTable(bRet, gaugeTableFile)
        Debug.Print("      Use gauge table? " & bRet)
        Debug.Print("      Error code as defined in swSheetMetalModifierError_e: " & lRet)

    End Sub

    Sub Process_SM3dBend _
    ( _
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
    )

```

```

)

Debug.Print(" +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

Dim swSketchBend As SketchedBendFeatureData
Dim swCustBend As CustomBendAllowance

swSketchBend = swFeat.GetDefinition
swCustBend = swSketchBend.GetCustomBendAllowance

Debug.Print("      UseDefaultBendAllowance = " & swSketchBend.UseDefaultBendAllowance)
Debug.Print("      UseDefaultBendRadius = " & swSketchBend.UseDefaultBendRadius)
Debug.Print("      BendRadius = " & swSketchBend.BendRadius * 1000.0# & " mm")

Process_CustomBendAllowance(swApp, swModel, swCustBend)

End Sub

Sub Process_SMMiteredFlange _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
)

    Debug.Print(" +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

    Dim swMiterFlange As MiterFlangeFeatureData
    swMiterFlange = swFeat.GetDefinition

    Debug.Print("      UseDefaultBendAllowance = " & swMiterFlange.UseDefaultBendAllowance)
    Debug.Print("      UseDefaultBendRadius = " & swMiterFlange.UseDefaultBendRadius)
    Debug.Print("      BendRadius = " & swMiterFlange.BendRadius * 1000.0# & " mm")

End Sub

Sub Process_Bends _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swBends As BendsFeatureData _
)

    Dim swCustBend As CustomBendAllowance
    swCustBend = swBends.GetCustomBendAllowance

    Debug.Print("      BendRadius = " & swBends.BendRadius * 1000.0# & " mm")
    Debug.Print("      UseDefaultBendAllowance = " & swBends.UseDefaultBendAllowance)
    Debug.Print("      UseDefaultBendRadius = " & swBends.UseDefaultBendRadius)

    Process_CustomBendAllowance(swApp, swModel, swCustBend)

End Sub

Sub Process_ProcessBends _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
)

    Debug.Print(" +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

    Dim swBends As BendsFeatureData
    swBends = swFeat.GetDefinition

    Process_Bends(swApp, swModel, swBends)

End Sub

Sub Process_FlattenBends _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
)

    Debug.Print(" +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

    Dim swBends As BendsFeatureData
    swBends = swFeat.GetDefinition

    Process_Bends(swApp, swModel, swBends)

End Sub

Sub Process_EdgeFlange _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
)

    Debug.Print(" +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

```

```

Dim swEdgeFlange As EdgeFlangeFeatureData
swEdgeFlange = swFeat.GetDefinition

Debug.Print("    UseDefaultBendRadius = " & swEdgeFlange.UseDefaultBendRadius)
Debug.Print("    BendRadius = " & swEdgeFlange.BendRadius * 1000.0# & " mm")

End Sub

Sub Process_FlatPattern _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
)

    Debug.Print("    +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

    Dim swFlatPatt As FlatPatternFeatureData
    swFlatPatt = swFeat.GetDefinition

    Debug.Print("    Simplify bends? " & swFlatPatt.SimplifyBends)

End Sub

Sub Process_Hem _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
)

    Debug.Print("    +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

    Dim swHem As HemFeatureData
    Dim swCustBend As CustomBendAllowance

    swHem = swFeat.GetDefinition
    swCustBend = swHem.GetCustomBendAllowance

    Debug.Print("    UseDefaultBendAllowance = " & swHem.UseDefaultBendAllowance)
    Debug.Print("    Radius = " & swHem.Radius * 1000.0# & " mm")

    Process_CustomBendAllowance(swApp, swModel, swCustBend)

End Sub

Sub Process_Jog _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
)

    Debug.Print("    +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

    Dim swJog As JogFeatureData
    Dim swCustBend As CustomBendAllowance

    swJog = swFeat.GetDefinition
    swCustBend = swJog.GetCustomBendAllowance

    Debug.Print("    UseDefaultBendAllowance = " & swJog.UseDefaultBendAllowance)
    Debug.Print("    UseDefaultBendRadius = " & swJog.UseDefaultBendRadius)
    Debug.Print("    BendRadius = " & swJog.BendRadius * 1000.0# & " mm")

    Process_CustomBendAllowance(swApp, swModel, swCustBend)

End Sub

Sub Process_LoftedBend _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
)

    Debug.Print("    +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

    Dim swLoftBend As LoftedBendsFeatureData
    swLoftBend = swFeat.GetDefinition

End Sub

Sub Process_Rip _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
)

```

```

    Debug.Print("  +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

    Dim swRip As RipFeatureData
    swRip = swFeat.GetDefinition

End Sub

Sub Process_CornerFeat _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
)

    Debug.Print("  +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

    Dim swCloseCorner As ClosedCornerFeatureData
    swCloseCorner = swFeat.GetDefinition

End Sub

Sub Process_OneBend _
(
    ByVal swApp As SldWorks, _
    ByVal swModel As ModelDoc2, _
    ByVal swFeat As Feature _
)

    Debug.Print("  +" & swFeat.Name & " [" & swFeat.GetTypeName & "]")

    Dim swOneBend As OneBendFeatureData
    Dim swCustBend As CustomBendAllowance

    swOneBend = swFeat.GetDefinition
    swCustBend = swOneBend.GetCustomBendAllowance

    Debug.Print("      UseDefaultBendAllowance = " & swOneBend.UseDefaultBendAllowance)
    Debug.Print("      UseDefaultBendRadius = " & swOneBend.UseDefaultBendRadius)

    Process_CustomBendAllowance(swApp, swModel, swCustBend)

End Sub

Sub main()

    swModel = swApp.ActiveDoc
    swSelMgr = swModel.SelectionManager
    swFeat = swModel.FirstFeature

    Debug.Print("File = " & swModel.GetPathName)

    Do While Not swFeat Is Nothing
        ' Process top-level sheet metal features
        Select Case swFeat.GetTypeName
            Case "SMBaseFlange"
                Process_SMBaseFlange(swApp, swModel, swFeat)

            Case "SheetMetal"
                Process_SheetMetal(swApp, swModel, swFeat)

            Case "SM3dBend"
                Process_SM3dBend(swApp, swModel, swFeat)

            Case "SMMiteredFlange"
                Process_SMMiteredFlange(swApp, swModel, swFeat)

            Case "ProcessBends"
                Process_ProcessBends(swApp, swModel, swFeat)

            Case "FlattenBends"
                Process_FlattenBends(swApp, swModel, swFeat)

            Case "EdgeFlange"
                Process_EdgeFlange(swApp, swModel, swFeat)

            Case "FlatPattern"
                Process_FlatPattern(swApp, swModel, swFeat)

            Case "Hem"
                Process_Hem(swApp, swModel, swFeat)

            Case "Jog"
                Process_Jog(swApp, swModel, swFeat)

            Case "LoftedBend"
                Process_LoftedBend(swApp, swModel, swFeat)

            Case "Rip"
                Process_Rip(swApp, swModel, swFeat)

            Case "CornerFeat"

```

```

        Process_CornerFeat(swApp, swModel, swFeat)

    Case Else
        ' Probably not a sheet metal feature
    End Select

' process sheet metal sub-features
swSubFeat = swFeat.GetFirstSubFeature

Do While Not swSubFeat Is Nothing
    Select Case swSubFeat.GetTypeName
        Case "OneBend"
            Process_OneBend(swApp, swModel, swSubFeat)

            Case Else
                ' Probably not a sheet metal feature
            End Select

        swSubFeat = swSubFeat.GetNextSubFeature()
    Loop

    swFeat = swFeat.GetNextFeature

Loop

End Sub

Public swApp As SldWorks

End Class

```

Get All Sketch Relations and Their Types Example (VBA)

This example shows how to get all of the sketch relations and their types for the selected sketch.

```
'-----
' Preconditions:
' 1. Open a part document.
' 2. Select a sketch.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets the sketch relations for the selected sketch.
' 2. Examine the Immediate window.
'-----
Option Explicit
```

```
Public Enum swSketchRelationFilterType_e
    swAll = 0
    swDangling = 1
    swOverDefining = 2
    swExternal = 3
    swDefinedInContext = 4
    swLocked = 5
    swBroken = 6
    swSelectedEntities = 7
End Enum
```

```
Public Enum swConstraintType_e
    swConstraintType_INVALIDIDTYPE = 0
    swConstraintType_DISTANCE = 1
    swConstraintType_ANGLE = 2
    swConstraintType_RADIUS = 3
    swConstraintType_HORIZONTAL = 4
    swConstraintType_VERTICAL = 5
    swConstraintType_TANGENT = 6
    swConstraintType_PARALLEL = 7
    swConstraintType_PERPENDICULAR = 8
    swConstraintType_COINCIDENT = 9
    swConstraintType_CONCENTRIC = 10
    swConstraintType_SYMMETRIC = 11
    swConstraintType_ATMIDDLE = 12
    swConstraintType_ATINTERSECT = 13
    swConstraintType_SAMELENGTH = 14
    swConstraintType_DIAMETER = 15
    swConstraintType_OFFSETEGE = 16
    swConstraintType_FIXED = 17
    swConstraintType_ARCANG90 = 18
    swConstraintType_ARCANG180 = 19
    swConstraintType_ARCANG270 = 20
    swConstraintType_ARCANGTOP = 21
    swConstraintType_ARCANGBOTTOM = 22
    swConstraintType_ARCANGLEFT = 23
    swConstraintType_ARCANGRIGHT = 24
    swConstraintType_HORIZPOINTS = 25
    swConstraintType_VERTPOINTS = 26
    swConstraintType_COLINEAR = 27
    swConstraintType_CORADIAL = 28
    swConstraintType_SNAPGRID = 29
    swConstraintType_SNAPLENGTH = 30
    swConstraintType_SNAPANGLE = 31
    swConstraintType_USEEDGE = 32
    swConstraintType_ELLIPSEANG90 = 33
    swConstraintType_ELLIPSEANG180 = 34
    swConstraintType_ELLIPSEANG270 = 35
    swConstraintType_ELLIPSEANGTOP = 36
    swConstraintType_ELLIPSEANGBOTTOM = 37
    swConstraintType_ELLIPSEANGLEFT = 38
    swConstraintType_ELLIPSEANGRIGHT = 39
    swConstraintType_ATPIERCE = 40
    swConstraintType_DOUBLEDISTANCE = 41
    swConstraintType_MERGEPOINTS = 42
    swConstraintType_ANGLE3P = 43
    swConstraintType_ARCLENGTH = 44
    swConstraintType_NORMAL = 45
    swConstraintType_NORMALPOINTS = 46
    swConstraintType_SKETCHOFFSET = 47
    swConstraintType_ALONGX = 48
    swConstraintType_ALONGY = 49
    swConstraintType_ALONGZ = 50
    swConstraintType_ALONGXPOINTS = 51
    swConstraintType_ALONGYPOINTS = 52
    swConstraintType_ALONGZPOINTS = 53
    swConstraintType_PARALLELYZ = 54
    swConstraintType_PARALLELZX = 55
    swConstraintType_INTERSECTION = 56
    swConstraintType_PATTERNEED = 57
    swConstraintType_ISOBYPOINT = 58
    swConstraintType_SAMEISOPARAM = 59
    swConstraintType_FITSPLINE = 60
End Enum

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
```

```
Dim swFeat As SldWorks.Feature
Dim swSketch As SldWorks.Sketch
Dim swSkRelMgr As SldWorks.SketchRelationManager
Dim vSkRelArr As Variant
Dim swSkRel As SldWorks.SketchRelation
Dim relType As Long
Dim i As Long

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swSelMgr = swModel.SelectionManager
Set swFeat = swSelMgr.GetSelectedObject5(1)
Set swSketch = swFeat.GetSpecificFeature2
Set swSkRelMgr = swSketch.RelationManager

vSkRelArr = swSkRelMgr.GetRelations(swAll)

For i = 0 To UBound(vSkRelArr)
    Set swSkRel = vSkRelArr(i)
    relType = swSkRel.GetRelationType
    Debug.Print relType
Next i

End Sub
```

Get All Sketch Segments in Drawing Template Example (VBA)

This example shows how to get all of the sketch segments in a drawing template.

```
'-----
' Preconditions:
' 1. Open a drawing.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets all of the sketch segments in the drawing
'    template and their type and length.
' 2. Examine the Immediate window.
'-----
Option Explicit

Public Enum swSketchSegments_e
    swSketchLINE = 0
    swSketchARC = 1
    swSketchELLIPSE = 2
    swSketchSPLINE = 3
    swSketchTEXT = 4
    swSketchPARABOLA = 5
End Enum

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swDraw As SldWorks.DrawingDoc
    Dim swView As SldWorks.View
    Dim swSketch As SldWorks.Sketch
    Dim swSketchSeg As SldWorks.SketchSegment
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swSelData As SldWorks.SelectData
    Dim vSketchSeg As Variant
    Dim vSketchSegID As Variant
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swSelData = swSelMgr.CreateSelectData
    Set swDraw = swModel

    swDraw.EditTemplate

    ' This is the drawing template
    Set swSketch = swModel.GetActiveSketch2

    Debug.Print "File = " & swModel.GetPathName

    swModel.ClearSelection2 (True)

    vSketchSeg = swSketch.GetSketchSegments
    For i = 0 To UBound(vSketchSeg)
        Set swSketchSeg = vSketchSeg(i)
        vSketchSegID = swSketchSeg.GetID

        Debug.Print "    SketchSegID(" & i & ") = [" & vSketchSegID(0) & ", " & vSketchSegID(1) & "]"
        Debug.Print "        Type      = " & swSketchSeg.GetType
        Debug.Print "        Length    = " & swSketchSeg.GetLength

        bRet = swSketchSeg.Select4(True, swSelData)
    Next

End Sub
```

Get All Visible Components in Drawing View Example (VBA)

This example shows how to get all of the visible components in a drawing view.

```
'-----
' Preconditions:
' 1. Open a drawing document that contains Drawing View1
'    with one or more components.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the components and their names in Drawing View1.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swView As SldWorks.View
Dim swComp As SldWorks.Component2
Dim vComps As Variant
Dim boolstatus As Boolean
Dim itr As Long

Sub main()

Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swModelDocExt = swModel.Extension
Set swSelMgr = swModel.SelectionManager

boolstatus = swModel.ActivateView("Drawing View1")
boolstatus = swModelDocExt.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0) w
Set swView = swSelMgr.GetSelectedObject6(1, -1)

swModel.ClearSelection2 True

'Get all visible components in the Drawing View1
vComps = swView.GetVisibleComponents

' Print the number of visible components
Debug.Print "Number of components:"
Debug.Print " " & UBound(vComps)
' Iterate through the visible components and print each one's name
Debug.Print "Names of components:"
For itr = 0 To UBound(vComps)
    Set swComp = vComps(itr)
    Debug.Print " " & swComp.Name2
Next itr

End Sub
```

Get and Fill Gaps in Body Example (C#)

This example shows how to get and fill the gaps in a body.

```
//-----
// Preconditions:
// 1. Open a model document that contains at least one body
// with one or more gaps.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Identifies the number of gaps in each
// body and fills each gap with a fill-surface
// feature.
// 2. Examine the Immediate window and the FeatureManager
// design tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace DiagnoseResultCSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            PartDoc swPart = default(PartDoc);
            object[] vBodyArr = null;
            Body2 swBody = default(Body2);
            int i = 0;
            int length = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swPart = (PartDoc)swModel;
            Debug.Print("File = " + swModel.GetPathName());
            for (i = 0; i <= 5; i++)
            {
                vBodyArr = (object[])swPart.GetBodies2(i, false);
                if (vBodyArr != null)
                {
                    length = vBodyArr.GetLength(0);
                    Debug.Print(" NumBody[" + i + "] = " + length.ToString());
                    foreach (Body2 vBody in vBodyArr)
                    {
                        swBody = vBody;
                        ProcessBody(swApp, swModel, swBody);
                    }
                }
            }
        }

        public void ProcessBody(SldWorks swApp, ModelDoc2 swModel, Body2 swBody)
        {
            int nRetVal1 = 0;
            int nRetVal2 = 0;
            DiagnoseResult swDiagnose = default(DiagnoseResult);
            object[] vCoEdgeArr = null;
            CoEdge swCoEdge = default(CoEdge);
            Edge swEdge = default(Edge);
            Entity swEnt = default(Entity);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SelectData swSelData = default(SelectData);
            FeatureManager swFeatMgr = default(FeatureManager);
            Feature swFeat = default(Feature);
            FaultEntity swFaultEnt = default(FaultEntity);
            int nNumGap = 0;
            int i = 0;
            bool bRet = false;

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swSelData = (SelectData)swSelMgr.CreateSelectData();
            swFeatMgr = (FeatureManager)swModel.FeatureManager;
            swDiagnose = (DiagnoseResult)swBody.Diagnose();
            swFaultEnt = swBody.Check3;
            nRetVal1 = swFaultEnt.Count;
            swFaultEnt = swBody.Check3;
            nRetVal2 = swFaultEnt.Count;
            swSelData.Mark = 257;
            nNumGap = swDiagnose.GetGapsCount();

            Debug.Print(" Body check1 = " + nRetVal1);
            Debug.Print(" Body check2 (Number of faults) = " + nRetVal2);
            Debug.Print(" Number of gaps = " + nNumGap);

            for (i = 1; i <= nNumGap; i++)
            {
                swModel.ClearSelection2(true);
                vCoEdgeArr = (object[])swDiagnose.GetCoEdgesAtGap(i - 1);
                foreach (CoEdge vCoEdge in vCoEdgeArr)
                {
                    swCoEdge = (CoEdge)vCoEdge;
                    swEdge = (Edge)swCoEdge.GetEdge();
                    swEnt = (Entity)swEdge;
                    bRet = swEnt.Select4(true, swSelData);
                }
                swFeat = (Feature)swFeatMgr.InsertFillSurface2(3, (int)swFeatureFillSurfaceOptions_e.swOptimizeSurface, swSelData, (int)swContactType_e.swContact, null, null);
            }
        }
    }
}
```

```
/// <summary>
///   The Sldworks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
```

Get and Fill Gaps in Body Example (VBA)

This example shows how to get and fill the gaps in a body.

```
'-----
' Preconditions:
' 1. Open a model document that contains at least one body
'    with one or more gaps.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Identifies the number of gaps in each
'    body and fills each gap with a fill-surface
'    feature.
' 2. Examine the Immediate window and the FeatureManager
'    design tree.
'-----
```

Option Explicit

Sub ProcessBody(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swBody As SldWorks.Body2)

```
Dim nRetVal1 As Long
Dim nRetVal2 As Long
Dim swDiagnose As SldWorks.DiagnoseResult
Dim vCoEdgeArr As Variant
Dim vCoEdge As Variant
Dim swCoEdge As SldWorks.CoEdge
Dim swEdge As SldWorks.Edge
Dim swEnt As SldWorks.Entity
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSelData As SldWorks.SelectData
Dim swFeatMgr As SldWorks.FeatureManager
Dim swFeat As SldWorks.Feature
Dim swFaultEnt As SldWorks.FaultEntity
Dim nNumGap As Long
Dim i As Long
Dim bRet As Boolean
```

```
Set swSelMgr = swModel.SelectionManager
Set swSelData = swSelMgr.CreateSelectData
Set swFeatMgr = swModel.FeatureManager
Set swDiagnose = swBody.Diagnose
Set swFaultEnt = swBody.Check3
nRetVal1 = swFaultEnt.Count
Set swFaultEnt = swBody.Check3
nRetVal2 = swFaultEnt.Count
swSelData.Mark = 257
nNumGap = swDiagnose.GetGapsCount
```

```
Debug.Print "    Body check1                = " & nRetVal1
Debug.Print "    Body check2 (Number of faults) = " & nRetVal2
Debug.Print "    Number of gaps                = " & nNumGap
```

```
For i = 1 To nNumGap
    swModel.ClearSelection2 True
    vCoEdgeArr = swDiagnose.GetCoEdgesAtGap(i - 1)
    For Each vCoEdge In vCoEdgeArr
        Set swCoEdge = vCoEdge
        Set swEdge = swCoEdge.GetEdge
        Set swEnt = swEdge
        bRet = swEnt.Select4(True, swSelData)
    Next vCoEdge
    Set swFeat = swFeatMgr.InsertFillSurface2(3, swOptimizeSurface, swSelData, swContact, Nothing, Nothing)
Next i
```

End Sub

Sub main()

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swPart As SldWorks.PartDoc
Dim vBodyArr As Variant
Dim vBody As Variant
Dim swBody As SldWorks.Body2
Dim i As Long
```

```
Set swApp = Application.SldWorks
Set swModel = swApp.ActiveDoc
Set swPart = swModel
Debug.Print "File = " & swModel.GetPathName
For i = 0 To 5
    vBodyArr = swPart.GetBodies2(i, False)
    If Not IsEmpty(vBodyArr) Then
        Debug.Print "    NumBody[" & i & "] = " & UBound(vBodyArr) + 1
        For Each vBody In vBodyArr
            Set swBody = vBody
            ProcessBody swApp, swModel, swBody
        Next vBody
    End If
Next i
```

End Sub

Get and Fill Gaps in Body Example (VB.NET)

This example shows how to get and fill the gaps in a body.

```

-----
Preconditions:
1. Open a model document that contains at least one body
   with one or more gaps.
2. Open the Immediate window.
Postconditions:
1. Identifies the number of gaps in each
   body and fills each gap with a fill-surface
   feature.
2. Examine the Immediate window and the FeatureManager
   design tree.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swPart As PartDoc
        Dim vBodyArr As Object
        Dim vBody As Object
        Dim swBody As Body2
        Dim i As Integer

        swModel = swApp.ActiveDoc
        swPart = swModel
        Debug.Print("File = " & swModel.GetPathName)
        For i = 0 To 5
            vBodyArr = swPart.GetBodies2(i, False)
            If Not vBodyArr Is Nothing Then
                Debug.Print(" NumBody[" & i & "] = " & UBound(vBodyArr) + 1)
                For Each vBody In vBodyArr
                    swBody = vBody
                    ProcessBody(swApp, swModel, swBody)
                Next vBody
            End If
        Next i
    End Sub

    Sub ProcessBody(ByVal swApp As SldWorks, ByVal swModel As ModelDoc2, ByVal swBody As Body2)
        Dim nRetVal1 As Integer
        Dim nRetVal2 As Integer
        Dim swDiagnose As DiagnoseResult
        Dim vCoEdgeArr As Object
        Dim vCoEdge As Object
        Dim swCoEdge As CoEdge
        Dim swEdge As Edge
        Dim swEnt As Entity
        Dim swSelMgr As SelectionMgr
        Dim swSelData As SelectData
        Dim swFeatMgr As FeatureManager
        Dim swFaultEnt As FaultEntity
        Dim swFeat As Feature
        Dim nNumGap As Integer
        Dim i As Integer
        Dim bRet As Boolean

        swSelMgr = swModel.SelectionManager
        swSelData = swSelMgr.CreateSelectData
        swFeatMgr = swModel.FeatureManager
        swDiagnose = swBody.Diagnose
        swFaultEnt = swBody.Check3
        nRetVal1 = swFaultEnt.Count
        swFaultEnt = swBody.Check3
        nRetVal2 = swFaultEnt.Count
        swSelData.Mark = 257
        nNumGap = swDiagnose.GetGapsCount

        Debug.Print(" Body check1 = " & nRetVal1)
        Debug.Print(" Body check2 (Number of faults) = " & nRetVal2)
        Debug.Print(" Number of gaps = " & nNumGap)

        For i = 1 To nNumGap
            swModel.ClearSelection2(True)
            vCoEdgeArr = swDiagnose.GetCoEdgesAtGap(i - 1)
            For Each vCoEdge In vCoEdgeArr
                swCoEdge = vCoEdge
                swEdge = swCoEdge.GetEdge
                swEnt = swEdge
                bRet = swEnt.Select4(True, swSelData)
            Next vCoEdge
            swFeat = swFeatMgr.InsertFillSurface2(3, swFeatureFillSurfaceOptions_e.swOptimizeSurface, swSelData, swContactType_e.swContact, Nothing, Nothing)
        Next i
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get and Save Routing Settings Example (C#)

This example shows how to get and save routing settings.

```
//-----
// Preconditions:
// 1. In SOLIDWORKS, click Tools > Add-Ins, select SOLIDWORKS Routing, and
//    click OK.
// 2. In the IDE, right-click the project, select Add Reference,
//    browse install_dir\api\redist, and click
//    SolidWorks.Interop.SWRoutingLib.dll.
// 3. Ensure that c:\temp exists.
// 4. Open an Immediate window.
//
// Postconditions:
// 1. Saves routing settings to c:\temp\routingSettings.sqy.
// 2. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.SWRoutingLib;
using System.Runtime.InteropServices;
namespace CreateRoutingSettings_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            RoutingSettings routingSettings = default(RoutingSettings);
            Boolean boolstatus = false;

            string wireNameHdr = null;
            string fromRefHdr = null;
            string fromPinHdr = null;
            string fromPartNoHdr = null;
            string toRefHdr = null;
            string toPinHdr = null;
            string toPartNoHdr = null;
            string cableNameHdr = null;
            string coreNameHdr = null;
            string colourHdr = null;
            string wireSpecHdr = null;
            string otherAttribHdr = null;
            wireNameHdr = "";
            fromRefHdr = "";
            fromPinHdr = "";
            fromPartNoHdr = "";
            toRefHdr = "";
            toPinHdr = "";
            toPartNoHdr = "";
            cableNameHdr = "";
            coreNameHdr = "";
            colourHdr = "";
            wireSpecHdr = "";
            otherAttribHdr = "";

            routingSettings = swApp.GetRoutingSettings();

            boolstatus = routingSettings.SaveRoutingSettingsToFile("c:\\temp\\routingSettings.sqy");
            boolstatus = routingSettings.GetFromToListHeaderDefinitions(out wireNameHdr, out fromRefHdr, out fromPinHdr, out fromPartNoHdr,
            out wireSpecHdr, out otherAttribHdr);
            Debug.Print("From-to list column headers");
            Debug.Print(" Wire name: " + wireNameHdr);
            Debug.Print(" From reference: " + fromRefHdr);
            Debug.Print(" From pin: " + fromPinHdr);
            Debug.Print(" From part number: " + fromPartNoHdr);
            Debug.Print(" To reference: " + toRefHdr);
            Debug.Print(" To pin: " + toPinHdr);
            Debug.Print(" To part number: " + toPartNoHdr);
            Debug.Print(" Cable name: " + cableNameHdr);
            Debug.Print(" Core name: " + coreNameHdr);
            Debug.Print(" Color: " + colourHdr);
            Debug.Print(" Wire spec: " + wireSpecHdr);
            Debug.Print(" Other attribute: " + otherAttribHdr);
            Debug.Print("");
            Debug.Print("Slack %: " + routingSettings.GetRoutingUserPreferenceDoubleValue((int)swUserPreferenceRoutingDouble_e.swSlack);
            Debug.Print("Text size for connection and route points: " + routingSettings.GetRoutingUserPreferenceDoubleValue((int)swUserPreferenceRoutingDouble_e.swTextSize);
            Debug.Print("Rotation increment in degrees: " + routingSettings.GetRoutingUserPreferenceIntegerValue((int)swUserPreferenceRoutingInteger_e.swRotationIncrement);
            Debug.Print("Enable minimum bend radius checks: " + routingSettings.GetRoutingUserPreferenceIntegerValue((int)swUserPreferenceRoutingInteger_e.swEnableMinimumBendRadiusChecks);
            Debug.Print("");
            Debug.Print("Default file locations");
            Debug.Print(" Assembly template: " + routingSettings.GetRoutingUserPreferenceStringValue((int)swUserPreferenceRoutingString_e.swAssemblyTemplate));
        }
    }
}
```

```

Debug.Print(" Cable library: " + routingSettings.GetRoutingUserPreferenceStringValue((int)swUserPreferenceRoutingFileLoca
Debug.Print(" Component library: " + routingSettings.GetRoutingUserPreferenceStringValue((int)swUserPreferenceRoutingFi
Debug.Print(" Covering library: " + routingSettings.GetRoutingUserPreferenceStringValue((int)swUserPreferenceRoutingFil
Debug.Print(" Interconnects library: " + routingSettings.GetRoutingUserPreferenceStringValue((int)swUserPreferenceRouti
Debug.Print(" Pipe tube covering library: " + routingSettings.GetRoutingUserPreferenceStringValue((int)swUserPreference
Debug.Print(" Standard cable: " + routingSettings.GetRoutingUserPreferenceStringValue((int)swUserPreferenceRoutingFilel
Debug.Print(" Standard tubes: " + routingSettings.GetRoutingUserPreferenceStringValue((int)swUserPreferenceRoutingFilel
Debug.Print(" Tag schemes: " + routingSettings.GetRoutingUserPreferenceStringValue((int)swUserPreferenceRoutingFileLoca
Debug.Print(" Routing library path: " + routingSettings.GetRoutingUserPreferenceStringValue((int)swUserPreferenceRoutir
Debug.Print("");
Debug.Print("Always use default document template? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPrefere
Debug.Print("Automatically add dimensions to route stubs? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUser
Debug.Print("Automatically create sketch fillets? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPreferer
Debug.Print("Automatically route on drop of clips? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPrefere
Debug.Print("Automatically route on drop of flange connectors? " + routingSettings.GetRoutingUserPreferenceToggle((int)s
Debug.Print("Create custom fittings? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPreferenceRoutingTogg
Debug.Print("Create pipes on open line segments? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPreference
Debug.Print("Display error balloons? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPreferenceRoutingTogg
Debug.Print("Enable route error checking? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPreferenceRoutir
Debug.Print("Include coverings in bills of material? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPrefe
Debug.Print("Save route assembly externally? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPreferenceRou
Debug.Print("Save route parts externally? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPreferenceRoutir
Debug.Print("Use automatic naming for route parts? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPrefere
Debug.Print("User centerline dimension? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUserPreferenceRouting1
Debug.Print("Use triad to position and orient components? " + routingSettings.GetRoutingUserPreferenceToggle((int)swUser

```

```

}

```

```

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

```

```

public SldWorks swApp;

```

```

}

```

```

}

```

Get and Save Routing Settings Example (VBA)

This example shows how to get and save routing settings.

```

'-----
' Preconditions:
' 1. In SOLIDWORKS, click Tools > Add-Ins, select SOLIDWORKS Routing, and
'    click OK.
' 2. In the IDE, click Tools > References and select
'    SOLIDWORKS version Routing Type Library.
' 3. Ensure that c:\temp exists.
' 4. Open an Immediate window.
'
' Postconditions:
' 1. Saves routing settings to c:\temp\routingSettings.sqy.
' 2. Inspect the Immediate window.
'-----

Dim swApp As SldWorks.SldWorks
Option Explicit

Sub main()

    Dim routingSettings As SldWorks.routingSettings
    Dim boolstatus As Long

    Dim wireNameHdr As String, fromRefHdr As String, fromPinHdr As String, fromPartNoHdr As String, toRefHdr As String, toPinHdr As
String, toPartNoHdr As String, cableNameHdr As String, coreNameHdr As String, colourHdr As String, wireSpecHdr As String,
otherAttribHdr As String
    wireNameHdr = ""
    fromRefHdr = ""
    fromPinHdr = ""
    fromPartNoHdr = ""
    toRefHdr = ""
    toPinHdr = ""
    toPartNoHdr = ""
    cableNameHdr = ""
    coreNameHdr = ""
    colourHdr = ""
    wireSpecHdr = ""
    otherAttribHdr = ""

    Set swApp = Application.SldWorks
    Set routingSettings = swApp.GetRoutingSettings

    boolstatus = routingSettings.SaveRoutingSettingsToFile("c:\temp\routingSettings.sqy")
    boolstatus = routingSettings.GetFromToListHeaderDefinitions(wireNameHdr, fromRefHdr, fromPinHdr, fromPartNoHdr, toRefHdr,
toPinHdr, toPartNoHdr, cableNameHdr, coreNameHdr, colourHdr, wireSpecHdr, otherAttribHdr)
    Debug.Print "From-to list column headers"
    Debug.Print " Wire name: " & wireNameHdr
    Debug.Print " From reference: " & fromRefHdr
    Debug.Print " From pin: " & fromPinHdr
    Debug.Print " From part number: " & fromPartNoHdr
    Debug.Print " To reference: " & toRefHdr
    Debug.Print " To pin: " & toPinHdr
    Debug.Print " To part number: " & toPartNoHdr
    Debug.Print " Cable name: " & cableNameHdr
    Debug.Print " Core name: " & coreNameHdr
    Debug.Print " Color: " & colourHdr
    Debug.Print " Wire spec: " & wireSpecHdr
    Debug.Print " Other attribute: " & otherAttribHdr
    Debug.Print ""
    Debug.Print "Slack %: " &
routingSettings.GetRoutingUserPreferenceDoubleValue(swUserPreferenceRoutingDouble_e.swSlackPercentage)
    Debug.Print "Text size for connection and route points: " &
routingSettings.GetRoutingUserPreferenceDoubleValue(swUserPreferenceRoutingDouble_e.swTextSizeForConnectionAndRoutePoints)
    Debug.Print "Rotation increment in degrees: " &
routingSettings.GetRoutingUserPreferenceIntegerValue(swUserPreferenceRoutingInteger_e.swComponentRotationIncrementDegrees)
    Debug.Print "Enable minimum bend radius checks: " &
routingSettings.GetRoutingUserPreferenceIntegerValue(swUserPreferenceRoutingInteger_e.swEnableMinimumBendRadiusChecks)
    Debug.Print ""
    Debug.Print "Default file locations"
    Debug.Print " Assembly template: " &
routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e.swFileLocationsRoutingAssemblyTemplate,
True)
    Debug.Print " Cable library: " &
routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e.swFileLocationsRoutingCableLibrary,
True)
    Debug.Print " Component library: " &
routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e.swFileLocationsRoutingComponentLibrary,
True)
    Debug.Print " Covering library: " &
routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e.swFileLocationsRoutingCoveringLibrary,
True)
    Debug.Print " Interconnects library: " &
routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e.swFileLocationsRoutingInterconnectsLibrar
True)
    Debug.Print " Pipe tube covering library: " &

```

```

routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e.swFileLocationsRoutingPipeTubeCoveringLit
True)
    Debug.Print "    Standard cable: " &
routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e.swFileLocationsRoutingStandardCable,
True)
    Debug.Print "    Standard tubes: " &
routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e.swFileLocationsRoutingStandardTubes,
True)
    Debug.Print "    Tag schemes: " &
routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e.swFileLocationsRoutingTagSchemes, True)
    Debug.Print "    Routing library path: " &
routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e.swRoutingLibraryPath, True)
    Debug.Print ""
    Debug.Print "Always use default document template? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swAlwaysUseDefaultDocumentTemplate)
    Debug.Print "Automatically add dimensions to route stubs? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swAutomaticallyAddDimensionToRouteStub)
    Debug.Print "Automatically create sketch fillets? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swAutomaticallyCreateSketchFillets)
    Debug.Print "Automatically route on drop of clips? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swAutomaticallyRouteOnDropOfClips)
    Debug.Print "Automatically route on drop of flange connectors? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swAutomaticallyRouteOnDropOfFlangeConnectors)
    Debug.Print "Create custom fittings? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swCreateCustomFittings)
    Debug.Print "Create pipes on open line segments? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swCreatePipesOnOpenLineSegments)
    Debug.Print "Display error balloons? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swDisplayErrorBalloons)
    Debug.Print "Enable route error checking? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swEnableRouteErrorChecking)
    Debug.Print "Include coverings in bills of material? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swIncludeCoveringsInBOM)
    Debug.Print "Save route assembly externally? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swSaveRouteAssemblyExternally)
    Debug.Print "Save route parts externally? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swSaveRoutePartsExternally)
    Debug.Print "Use automatic naming for route parts? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swUseAutomaticNamingForRouteParts)
    Debug.Print "User centerline dimension? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swUseCenterlineDim)
    Debug.Print "Use triad to position and orient components? " &
routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swUseTriadToPosAndOrientComp)

```

End Sub

Get and Save Routing Settings Example (VB.NET)

This example shows how to get and save routing settings.

```

'-----
' Preconditions:
' 1. In SOLIDWORKS, click Tools > Add-Ins, select SOLIDWORKS Routing, and
'    click OK.
' 2. In the IDE, right-click the project, select Add Reference,
'    browse install_dir\api\redist, and click
'    SolidWorks.Interop.SWRoutingLib.dll.
' 3. Ensure that c:\temp exists.
' 4. Open an Immediate window.
'-----
' Postconditions:
' 1. Saves routing settings to c:\temp\routingSettings.sqy.
' 2. Inspect the Immediate window.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.SWRoutingLib
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim routingSettings As RoutingSettings
        Dim boolstatus As Integer

        Dim wireNameHdr As String, fromRefHdr As String, fromPinHdr As String, fromPartNoHdr As String, toRefHdr As String, toPinHdr
        wireNameHdr = ""
        fromRefHdr = ""
        fromPinHdr = ""
        fromPartNoHdr = ""
        toRefHdr = ""
        toPinHdr = ""
        toPartNoHdr = ""
        cableNameHdr = ""
        coreNameHdr = ""
        colourHdr = ""
        wireSpecHdr = ""
        otherAttribHdr = ""

        routingSettings = swApp.GetRoutingSettings

        boolstatus = routingSettings.SaveRoutingSettingsToFile("c:\temp\routingSettings.sqy")
        boolstatus = routingSettings.GetFromToListHeaderDefinitions(wireNameHdr, fromRefHdr, fromPinHdr, fromPartNoHdr, toRefHdr, to
        Debug.Print("From-to list column headers")
        Debug.Print(" Wire name: " & wireNameHdr)
        Debug.Print(" From reference: " & fromRefHdr)
        Debug.Print(" From pin: " & fromPinHdr)
        Debug.Print(" From part number: " & fromPartNoHdr)
        Debug.Print(" To reference: " & toRefHdr)
        Debug.Print(" To pin: " & toPinHdr)
        Debug.Print(" To part number: " & toPartNoHdr)
        Debug.Print(" Cable name: " & cableNameHdr)
        Debug.Print(" Core name: " & coreNameHdr)
        Debug.Print(" Color: " & colourHdr)
        Debug.Print(" Wire spec: " & wireSpecHdr)
        Debug.Print(" Other attribute: " & otherAttribHdr)
        Debug.Print("")
        Debug.Print("Slack %: " & routingSettings.GetRoutingUserPreferenceDoubleValue(swUserPreferenceRoutingDouble_e.swSlackPercent
        Debug.Print("Text size for connection and route points: " & routingSettings.GetRoutingUserPreferenceDoubleValue(swUserPrefer
        Debug.Print("Rotation increment in degrees: " & routingSettings.GetRoutingUserPreferenceIntegerValue(swUserPreferenceRouting
        Debug.Print("Enable minimum bend radius checks: " & routingSettings.GetRoutingUserPreferenceIntegerValue(swUserPreferenceRoi
        Debug.Print("")
        Debug.Print("Default file locations")
        Debug.Print(" Assembly template: " & routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocati
        Debug.Print(" Cable library: " & routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e
        Debug.Print(" Component library: " & routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocati
        Debug.Print(" Covering library: " & routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocati
        Debug.Print(" Interconnects library: " & routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLoc
        Debug.Print(" Pipe tube covering library: " & routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFi
        Debug.Print(" Standard cable: " & routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_
        Debug.Print(" Standard tubes: " & routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_
        Debug.Print(" Tag schemes: " & routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLocations_e.s
        Debug.Print(" Routing library path: " & routingSettings.GetRoutingUserPreferenceStringValue(swUserPreferenceRoutingFileLoca
        Debug.Print("")
        Debug.Print("Always use default document template? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutir
        Debug.Print("Automatically add dimensions to route stubs? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenc
        Debug.Print("Automatically create sketch fillets? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRouting
        Debug.Print("Automatically route on drop of clips? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutir
        Debug.Print("Automatically route on drop of flange connectors? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPref
        Debug.Print("Create custom fittings? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swCr
        Debug.Print("Create pipes on open line segments? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRouting
    
```

```

Debug.Print("Display error balloons? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.swDi
Debug.Print("Enable route error checking? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e
Debug.Print("Include coverings in bills of material? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRout
Debug.Print("Save route assembly externally? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingTogg
Debug.Print("Save route parts externally? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e
Debug.Print("Use automatic naming for route parts? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutir
Debug.Print("User centerline dimension? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenceRoutingToggle_e.s
Debug.Print("Use triad to position and orient components? " & routingSettings.GetRoutingUserPreferenceToggle(swUserPreferenc

```

End Sub

```

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Get and Set Blocks Data in Any Document Example (VBA)

This example shows how to get blocks data in part, assembly, and drawing documents. It also shows how to change the angles of the blocks.

```
'-----  
' Preconditions: Open a part, assembly, or drawing document containing  
' one or more block definitions and block instances.  
'  
' Postconditions: Modifies the angles of all block instances.  
'-----  
  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim skMgr As SldWorks.SketchManager  
Dim pBlock As SldWorks.SketchBlockDefinition  
Dim pInst As SldWorks.SketchBlockInstance  
Dim leaderPt As SldWorks.MathPoint  
Dim insPt As SldWorks.MathPoint  
Dim vInstPt As Variant  
Dim vInstances As Variant  
Dim vBlocks As Variant  
Dim nInstanceCount As Long  
Dim itr As Long  
Dim jtr As Long  
Dim bLinkToFile As Boolean  
Dim strInsPoint As String  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set skMgr = swModel.SketchManager  
  
    ' To change the angles, you must edit the block's sketches  
    swModel.EditSketch  
  
    vBlocks = skMgr.GetSketchBlockDefinitions  
  
    ' Exit if no blocks  
    If IsEmpty(vBlocks) Then  
        Exit Sub  
    End If  
  
    ' Process block definitions  
    For itr = 0 To UBound(vBlocks)  
  
        Set pBlock = vBlocks(itr)  
  
        ' Block definition linked to file?  
        Debug.Print "  Link To File = " & pBlock.LinkToFile  
  
        ' Block linked file name  
        Debug.Print "  Link File Name = " & pBlock.FileName  
  
        ' Block definition insertion point  
        Set insPt = pBlock.InsertionPoint  
        vInstPt = insPt.ArrayData  
        strInsPoint = "  Insertion point: x = " & CStr(vInstPt(0) * 1000) & " , y = " & CStr(vInstPt(1) * 1000) & " , z = " & CStr(vInstPt(2) * 1000)  
        Debug.Print strInsPoint  
  
        ' Number of block instances of this block definition  
        nInstanceCount = pBlock.GetInstanceCount  
        Debug.Print "  Instance Count = " & nInstanceCount  
  
        ' Process block instances  
        If nInstanceCount > 0 Then  
            vInstances = pBlock.GetInstances  
            For jtr = 0 To UBound(vInstances)
```

```
Set plnst = vinstances(jtr)
```

```
' Block instance position
```

```
Set insPt = plnst.InstancePosition
```

```
vInstPt = insPt.ArrayData
```

```
strInsPoint = "      Instance position: x = " + CStr(vInstPt(0) * 1000) + " , y = " + CStr(vInstPt(1) * 1000) + " , z = " + CStr(vInstPt(2) * 1000)
```

```
Debug.Print strInsPoint
```

```
' Get block instance angle
```

```
Debug.Print "      Original angle = " & plnst.Angle
```

```
' Change block instance angle
```

```
' 1 radian = 180°/p = 57.295779513° or approximately 57.3°
```

```
plnst.Angle = 90 / 57.3
```

```
Debug.Print "      Modified angle = " & plnst.Angle
```

```
' Block instance scale
```

```
Debug.Print "      Scale = " & plnst.Scale
```

```
' Block instance owner's sketch name
```

```
Debug.Print "      Owner Sketch = " & plnst.GetSketch.Name
```

```
Next jtr
```

```
End If
```

```
Next itr
```

```
End Sub
```

Get and Set Center Mark Set Example (C#)

This example shows how to get and set properties of a center mark set.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\introsw\pressure_plate.sldprt.
// 2. Click File > Make Drawing from Part > OK.
// 3. Drag Bottom from the View Palette to Sheet1 and click OK.
// 4. Click a center mark and press Delete.
// 5. Open the Immediate window.
//
// Postconditions:
// 1. Activates Drawing View1.
// 2. Selects a center mark in Drawing View1.
// 3. Gets and sets properties of the center mark set.
// 4. Examine the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            DrawingDoc swDrawingDoc = default(DrawingDoc);
            CenterMark swCenterMark = default(CenterMark);
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            bool status = false;
            int nbrInGroup = 0;
            int i = 0;
            double[] cmCoordinates = new double[3];

            swDrawingDoc = (DrawingDoc)swApp.ActiveDoc;

            //Get drawing view
            swDrawingDoc.ActivateView("Drawing View1");
            swModel = (ModelDoc2)swDrawingDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            //Select a center mark
            status = swModelDocExt.SelectByID("DetailItem217@Drawing View1", "CENTERMARKSYM", 0.650520142546246, 0.417728685527748, 0, false, 0, null, 0);

            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swCenterMark = (CenterMark)swSelectionMgr.GetSelectedObject6(1, -1);

            //Get and set properties of center mark set
            Debug.Print("Center mark set");
            Debug.Print(" Style (4 = swCenterMark_CircularGroup): " + swCenterMark.Style);
            Debug.Print(" Is grouped? " + swCenterMark.IsGrouped);
            nbrInGroup = swCenterMark.GroupCount;
            Debug.Print(" Number in set: " + (nbrInGroup - 1));
            Debug.Print(" Any detached? " + swCenterMark.HasDetachCenterMark());
            for (i = 0; i <= nbrInGroup - 1; i++)
            {
                Debug.Print(" Center mark: " + i);
                Debug.Print(" Detached: " + swCenterMark.IsDetached(i));
                cmCoordinates = (double[])swCenterMark.GetPosition(i);
                Debug.Print(" X, Y, Z coordinates: " + cmCoordinates[0] + " " + cmCoordinates[1] + " " + cmCoordinates[2]);
                Debug.Print(" Original extended length: " + swCenterMark.GetExtendedLength(i, (int)swCenterMarkHandle_e.swCenterMarkHandle_Right));
                status = swCenterMark.SetExtendedLength(i, (int)swCenterMarkHandle_e.swCenterMarkHandle_Right, 0.005);
                Debug.Print(" Modified extended length: " + swCenterMark.GetExtendedLength(i, (int)swCenterMarkHandle_e.swCenterMarkHandle_Right));
                Debug.Print(" Is deleted? " + swCenterMark.IsDeleted(i));
            }
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Get and Set Center Mark Set Example (VBA)

This example shows how to get and set properties of a center mark set.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\introsw\pressure_plate.sldprt.
' 2. Click File > Make Drawing from Part > OK.
' 3. Drag Bottom from the View Palette to Sheet1 and click OK.
' 4. Click a center mark and press Delete.
' 5. Open the Immediate window.
'
' Postconditions:
' 1. Activates Drawing View1.
' 2. Selects a center mark in Drawing View1.
' 3. Gets and sets properties of the center mark set.
' 4. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swDrawingDoc As SldWorks.DrawingDoc
Dim swCenterMark As SldWorks.CenterMark
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim status As Boolean
Dim nbrInGroup As Long
Dim i As Long
Dim cmCoordinates As Variant

Sub main()

    Set swApp = Application.SldWorks
    Set swDrawingDoc = swApp.ActiveDoc

    'Get drawing view
    swDrawingDoc.ActivateView ("Drawing View1")
    Set swModel = swDrawingDoc
    Set swModelDocExt = swModel.Extension

    'Select a center mark
    status = swModelDocExt.SelectByID("DetailItem217@Drawing View1", "CENTERMARKSYM", 0.434357613710555, 0.467612346028292, 0, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swCenterMark = swSelectionMgr.GetSelectedObject6(1, -1)

    'Get and set properties of center mark set
    Debug.Print "Center mark set"
    Debug.Print " Style (4 = swCenterMark_CircularGroup): " & swCenterMark.Style
    Debug.Print " Is grouped? " & swCenterMark.IsGrouped
    nbrInGroup = swCenterMark.GroupCount
    Debug.Print " Number in set: " & nbrInGroup - 1
    Debug.Print " Any detached? " & swCenterMark.HasDetachCenterMark
    For i = 0 To nbrInGroup - 1
        Debug.Print " Center mark: " & i
        Debug.Print " Detached: " & swCenterMark.IsDetached(i)
        cmCoordinates = swCenterMark.GetPosition(i)
        Debug.Print " x, y, z coordinates: " & cmCoordinates(0) & ", " & cmCoordinates(1) & ", " & cmCoordinates(2)
        Debug.Print " Original extended length: " & swCenterMark.GetExtendedLength(i, swCenterMarkHandle_e.swCenterMarkHandle_Right)
        status = swCenterMark.SetExtendedLength(i, swCenterMarkHandle_e.swCenterMarkHandle_Right, 0.005)
        Debug.Print " Modified extended length: " & swCenterMark.GetExtendedLength(i, swCenterMarkHandle_e.swCenterMarkHandle_Right)
        Debug.Print " Is deleted? " & swCenterMark.IsDeleted(i)
    Next i

End Sub
```

Get and Set Center Mark Set Example (VB.NET)

This example shows how to get and set properties of a center mark set.

```

-----
Preconditions:
1. Open public documents\samples\tutorial\introsw\pressure_plate.sldprt.
2. Click File > Make Drawing from Part > OK.
3. Drag Bottom from the View Palette to Sheet1 and click OK.
4. Click a center mark and press Delete.
5. Open the Immediate window.

Postconditions:
1. Activates Drawing View1.
2. Selects a center mark in Drawing View1.
3. Gets and sets properties of the center mark set.
4. Examine the Immediate window.

NOTE: Because the part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swDrawingDoc As DrawingDoc
        Dim swCenterMark As CenterMark
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelectionMgr As SelectionMgr
        Dim status As Boolean
        Dim nbrInGroup As Integer
        Dim i As Integer
        Dim cmCoordinates(2) As Double

        swDrawingDoc = swApp.ActiveDoc

        'Get drawing view
        swDrawingDoc.ActivateView("Drawing View1")
        swModel = swDrawingDoc
        swModelDocExt = swModel.Extension

        'Select a center mark
        status = swModelDocExt.SelectByID("DetailItem217@Drawing View1", "CENTERMARKSYM", 0.434357613710555, 0.467612346028292, 0, False, 0, Nothing, 0)
        swSelectionMgr = swModel.SelectionManager
        swCenterMark = swSelectionMgr.GetSelectedObject6(1, -1)

        'Get and set properties of center mark set
        Debug.Print("Center mark set")
        Debug.Print("Style (4 = swCenterMark.CircularGroup): " & swCenterMark.Style)
        Debug.Print("Is grouped? " & swCenterMark.IsGrouped)
        nbrInGroup = swCenterMark.GroupCount
        Debug.Print("Number in set: " & nbrInGroup - 1)
        Debug.Print("Any detached? " & swCenterMark.HasDetachCenterMark)
        For i = 0 To nbrInGroup - 1
            Debug.Print("Center mark: " & i)
            Debug.Print("Detached: " & swCenterMark.IsDetached(i))
            cmCoordinates = swCenterMark.GetPosition(i)
            Debug.Print("x, y, z coordinates: " & cmCoordinates(0) & ", " & cmCoordinates(1) & ", " & cmCoordinates(2))
            Debug.Print("Original extended length: " & swCenterMark.GetExtendedLength(i, swCenterMarkHandle_e.swCenterMarkHandle_Right))
            status = swCenterMark.SetExtendedLength(i, swCenterMarkHandle_e.swCenterMarkHandle_Right, 0.005)
            Debug.Print("Modified extended length: " & swCenterMark.GetExtendedLength(i, swCenterMarkHandle_e.swCenterMarkHandle_Right))
            Debug.Print("Is deleted? " & swCenterMark.IsDeleted(i))
        Next i
    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

End Class

Get and Set BOM Column Types and Cell Equations Example (VBA)

This example shows how to get and set BOM table column types and cell equations.

```

'-----
' Preconditions:
' 1. Ensure the specified drawing and BOM template exist.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens a document and inserts a BOM table annotation.
' 2. Inserts three rows at the bottom of the table.
' 3. Populates the first three rows of the Description column with
'    10, 20, and 30, respectively.
' 4. Inserts multiple columns of various types.
' 5. Sets equation, SUM(C1:C3), in cell(4,2), gets it, and evaluates it.
' 6. Examine the BOM table and the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim boolstatus As Boolean
Dim SelMgr As SldWorks.SelectionMgr
Dim theTableAnnotation As SldWorks.TableAnnotation
Dim SelObjType As Long
Dim TableAnnotationType As Long
Dim PropData As Variant
Dim longstatus As Long, longwarnings As Long

Sub main()
    Dim property_string As String
    Dim cellequation As String
    Dim solvedval As String

    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2019\samples\tutorial\advdrawings\foodprocessor.slddrw", 3, 0, "", longstatus, longwarnings)
    Dim swDrawing As SldWorks.DrawingDoc
    Set swDrawing = swModel
    swApp.ActivateDoc2 "foodprocessor - Sheet1", False, longstatus
    Set swModel = swApp.ActiveDoc
    boolstatus = swModel.ActivateView("Drawing View1")
    Dim swActiveView As SldWorks.View
    Set swActiveView = swModel.ActiveDrawingView
    Dim swBOMTableAnno As SldWorks.BomTableAnnotation
    Set swBOMTableAnno = swActiveView.InsertBomTable4(False, 0.111517224639408, 0.36564824,
swBOMConfigurationAnchorType_e.swBOMConfigurationAnchor_TopLeft, swBomType_e.swBomType_PartsOnly, "Default", "C:\Program
Files\SOLIDWORKS Corp\SOLIDWORKS\lang\english\bom-standard.sldbomtbt", False, 0, False)
    boolstatus = swModel.EditRebuild3()

    Set SelMgr = swModel.SelectionManager
    boolstatus = swModel.Extension.SelectByID2("DetailItem174@Sheet1", "ANNOTATIONTABLES", 0.196194597542634, 0.368489762580645, 0,
False, 0, Nothing, 0)
    SelObjType = SelMgr.GetSelectedObjectType3(1, -1)
    If SelObjType <> swSelANNOTATIONTABLES Then
        MsgBox "Select a BOM table in the drawing before running this example."
    End
    End If

    Set theTableAnnotation = SelMgr.GetSelectedObject6(1, -1)
    TableAnnotationType = theTableAnnotation.Type
    If TableAnnotationType <> swTableAnnotation_BillofMaterials Then
        MsgBox "Select a BOM table in the drawing before running this example."
    End
    End If

    Debug.Print " "
    Debug.Print " -----"
    Debug.Print "Table BEFORE inserting a column..."
    Debug.Print " "

    ' Display the table before inserting a column

    DisplayTableColumnProps theTableAnnotation

    ' Insert Custom Property column
    boolstatus = theTableAnnotation.InsertColumn2(swTableItemInsertPosition_Last, 0, "New Col Custom Property",
swInsertColumn_DefaultWidth)
    property_string = "Sw-Comments(Comments)"
    PropData = property_string
    longstatus = theTableAnnotation.SetColumnType3(theTableAnnotation.ColumnCount - 1, swBomTableColumnType_CustomProperty, True,

```

```

PropData)

    ' Insert Unit Of Measure column
    boolstatus = theTableAnnotation.InsertColumn2(swTableItemInsertPosition_Last, 0, "New Col Unit of Measure",
swInsertColumn_DefaultWidth)
    property_string = "SW-Title(Title)"
    PropData = property_string
    longstatus = theTableAnnotation.SetColumnType3(theTableAnnotation.ColumnCount - 1, swBomTableColumnType_UnitOfMeasure, True,
PropData)

    ' Insert Equation column
    boolstatus = theTableAnnotation.InsertColumn2(swTableItemInsertPosition_Last, 0, "New Col Equation",
swInsertColumn_DefaultWidth)
    property_string = "=2+2"
    PropData = property_string
    longstatus = theTableAnnotation.SetColumnType3(theTableAnnotation.ColumnCount - 1, swBomTableColumnType_Equation, True,
PropData)

    ' Insert Component Reference column
    boolstatus = theTableAnnotation.InsertColumn2(swTableItemInsertPosition_Last, 0, "New Col Component Ref",
swInsertColumn_DefaultWidth)
    Set PropData = Nothing
    longstatus = theTableAnnotation.SetColumnType3(theTableAnnotation.ColumnCount - 1, swBomTableColumnType_ComponentReference,
True, PropData)

    ' Insert Toolbox Property column
    boolstatus = theTableAnnotation.InsertColumn2(swTableItemInsertPosition_Last, 0, "New Col toolbox",
swInsertColumn_DefaultWidth)
    PropData = swToolBoxPropertyName_e.swToolBoxPropertyName_Specification
    longstatus = theTableAnnotation.SetColumnType3(theTableAnnotation.ColumnCount - 1, swBomTableColumnType_ToolboxProperty, True,
PropData)

    ' Insert Cut List Properties column -- only applicable for sheet metal parts
    boolstatus = theTableAnnotation.InsertColumn2(swTableItemInsertPosition_Last, 0, "New Col Cut List property",
swInsertColumn_DefaultWidth)
    property_string = "Bends"
    PropData = property_string
    longstatus = theTableAnnotation.SetColumnType3(theTableAnnotation.ColumnCount - 1, swBomTableColumnType_CutListProperties,
True, PropData)

    ' Insert Item Number column
    boolstatus = theTableAnnotation.InsertColumn2(swTableItemInsertPosition_Last, 0, "New Col Item Number",
swInsertColumn_DefaultWidth)
    PropData = Array(2, 3, True, False)
    longstatus = theTableAnnotation.SetColumnType3(theTableAnnotation.ColumnCount - 1, swBomTableColumnType_ItemNumber, True,
PropData)

    ' Insert Part Number column
    boolstatus = theTableAnnotation.InsertColumn2(swTableItemInsertPosition_Last, 0, "New Col Part Number",
swInsertColumn_DefaultWidth)
    PropData = True ' use title summary
    longstatus = theTableAnnotation.SetColumnType3(theTableAnnotation.ColumnCount - 1, swBomTableColumnType_PartNumber, True,
PropData)

    boolstatus = theTableAnnotation.InsertRow(4, 0)
    theTableAnnotation.Text2(1, 2, True) = 10
    boolstatus = theTableAnnotation.InsertRow(4, 0)
    theTableAnnotation.Text2(2, 2, True) = 20
    boolstatus = theTableAnnotation.InsertRow(4, 0)
    theTableAnnotation.Text2(3, 2, True) = 30

    Debug.Print " "
    Debug.Print " -----"
    Debug.Print "Table AFTER inserting a column..."
    Debug.Print " "

    ' Display table after inserting a column
    DisplayTableColumnProps theTableAnnotation

    ' Set a cell equation
    longstatus = theTableAnnotation.SetCellEquation(4, 2, True, "=SUM(C2:C4)")
    boolstatus = swModel.EditRebuild3()

    ' Get the cell equation
    cellequation = theTableAnnotation.GetCellEquation(4, 2, True, solvedval, longstatus)
    Debug.Print " "
    Debug.Print "Equation set in cell(4,2): " & cellequation

    ' Evaluate the cell equation
    longstatus = theTableAnnotation.EvaluateCellEquation(4, 2, True, cellequation, solvedval)
    Debug.Print " Solved value: " & solvedval

End Sub
Sub DisplayTableColumnProps(theTableAnnotation As TableAnnotation)
    Dim ColCount As Long
    Dim i As Long
    Dim ColType As swTableColumnTypes_e
    Dim ColTitle As String
    Dim status As Long

```

```

Dim property_string As String
Dim toolbox_property As Long
Dim start As Long
Dim increment As Long
Dim order_balloons As Boolean
Dim BOM_items As Boolean
Dim titlesummary As Boolean

Debug.Print "Col#"; vbTab; "Type"; vbTab; "Title"
ColCount = theTableAnnotation.ColumnCount
For i = 0 To ColCount - 1

    ColType = theTableAnnotation.GetColumnType3(i, True, PropData, status)
    ColTitle = theTableAnnotation.GetColumnTitle2(i, True)

    Debug.Print i; vbTab; ColType; vbTab; ColTitle; vbTab

    If ColType = swTableColumnTypes_e.swBomTableColumnType_CustomProperty Then
        property_string = PropData
        Debug.Print "                Property Name: " & property_string

    ElseIf ColType = swTableColumnTypes_e.swBomTableColumnType_UnitOfMeasure Then
        property_string = PropData
        Debug.Print "                Property Name: " & property_string

    ElseIf ColType = swTableColumnTypes_e.swBomTableColumnType_Equation Then
        property_string = PropData
        Debug.Print "                Equation      : " & property_string

    ElseIf ColType = swTableColumnTypes_e.swBomTableColumnType_ComponentReference Then
        ' PropData should be Nothing since a
        ' component reference column does not contain additional property data
        If PropData Is Nothing Then
            Debug.Print "                NULL Property data"
        End If

    ElseIf ColType = swTableColumnTypes_e.swBomTableColumnType_ToolboxProperty Then
        toolbox_property = PropData
        Debug.Print "                Toolbox Property: " & toolbox_property

    ElseIf ColType = swTableColumnTypes_e.swBomTableColumnType_CutListProperties Then
        property_string = PropData
        Debug.Print "                Cut List Property: " & property_string

    ElseIf ColType = swTableColumnTypes_e.swBomTableColumnType_ItemNumber Then
        start = PropData(0)
        increment = PropData(1)
        order_balloons = PropData(2)
        BOM_items = PropData(3)
        Debug.Print "                Start      : " & start
        Debug.Print "                Increment: " & increment
        Debug.Print "                Order balloons and Bill of Materials to follow assembly order: " & order_balloons
        Debug.Print "                Do not change Bill of Materials item numbers: " & BOM_items

    ElseIf ColType = swTableColumnTypes_e.swBomTableColumnType_PartNumber Then
        titlesummary = PropData
        Debug.Print "                Use title summary: " & titlesummary

    End If

Next i
End Sub

```

Get and Set Component's Suppression State Example (VBA)

This example shows how to get and set the suppression state of a fully resolved component in an assembly.

```
'-----
' Preconditions:
' 1. Open an assembly.
' 2. Select a component.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Sets the selected component's suppression state to
'    lightweight if the component is resolved.
' 2. Examine the Immediate window and the component in the
'    the FeatureManager design tree.
'-----
Option Explicit

Sub main()

    Dim swApp                As SldWorks.SldWorks
    Dim swModel              As SldWorks.ModelDoc2
    Dim swSelMgr             As SldWorks.SelectionMgr
    Dim swComp              As SldWorks.Component2
    Dim bRet                As Boolean
    Dim nRetVal              As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swComp = swSelMgr.GetSelectedObjectsComponent4(1, -1)

    Debug.Print "File = " & swModel.GetPathName
    Debug.Print "  Comp = " & swComp.Name2
    Debug.Print "    Path = " & swComp.GetPathName
    Debug.Print "  Suppression state as defined in swComponentSuppressionState_e = " & swComp.GetSuppression2
    Debug.Print "    Is suppressed? " & swComp.IsSuppressed

    nRetVal = swComp.SetSuppression2(swComponentSuppressionState_e.swComponentFullyLightweight)
    Debug.Print "    Is set lightweight (2 = state was changed) = " & nRetVal

End Sub
```

Get and Set Connection Point Feature Example (C#)

This example shows how to get and set data for connection point features.

```
//-----
// Preconditions:
// 1. Ensure that the latest SOLIDWORKS Design Library is loaded.
// 2. Drag and drop design library > routing > electrical > db9 male.sldprt
//    into the main view.
// 3. Observe CPoint1 in the FeatureManager design tree.
// 4. Run (F5) this macro.

// Postconditions:
// 1. Inspect the Immediate Window.
// 2. CPoint1 is now MyCPoint in the FeatureManager design tree.
//
// NOTE: Because this is a Design Library part, close it without saving changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace CPoint2_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 Part = default(ModelDoc2);
            SelectionMgr selMgr = default(SelectionMgr);
            int selCount = 0;
            int selType = 0;
            object selObj = null;
            Feature selFeat = null;
            object vLocation = null;
            double xLoc = 0;
            double yLoc = 0;
            double zLoc = 0;
            object vDirection = null;
            double xDir = 0;
            double yDir = 0;
            double zDir = 0;
            bool boolstatus = false;

            Part = (ModelDoc2)swApp.ActiveDoc;
            selMgr = (SelectionMgr)Part.SelectionManager;

            boolstatus = Part.Extension.SelectByID2("CPoint1", "CONNECTIONPOINT", 0, 0, 0, false, 0, null, 0);

            selCount = selMgr.GetSelectedObjectCount2(-1);
            if ((selCount > 0))
            {
                selType = selMgr.GetSelectedObjectType3(1, -1);
                selObj = selMgr.GetSelectedObject6(1, -1);
                if ((selType == (long)swSelectType_e.swSelCONNECTIONPOINTS))
                {
                    selFeat = (Feature)selObj;
                }
            }

            Part.ClearSelection2(true);

            object featData = null;
            if ((selFeat != null))
            {
                featData = selFeat.GetDefinition();
            }
            IConnectionPointFeatureData cPointData = default(IConnectionPointFeatureData);
            cPointData = (IConnectionPointFeatureData)featData;

            if ((cPointData != null))
            {
                Debug.Print("Get stublength = " + cPointData.StubLength);
                cPointData.StubLength = 0.009;
                Debug.Print("Set stublength = " + cPointData.StubLength);

                Debug.Print("Get diameter = " + cPointData.RouteDiameter);
                cPointData.RouteDiameter = 0.011;
                Debug.Print("Set diameter = " + cPointData.RouteDiameter);

                Debug.Print("Get route type = " + cPointData.RouteType);
                cPointData.RouteType = 2;
                Debug.Print("Set route type = " + cPointData.RouteType);

                Debug.Print("Get route sub type = " + cPointData.RouteSubType);
            }
        }
    }
}
```

```

cPointData.RouteSubType = 3;
Debug.Print("Set route sub type = " + cPointData.RouteSubType);

Debug.Print("Get electrical Pin ID = " + cPointData.ElectricalPinID);
cPointData.ElectricalPinID = "zxc";
Debug.Print("Set electrical Pin ID = " + cPointData.ElectricalPinID);

Debug.Print("Get port ID = " + cPointData.PortID);
cPointData.PortID = "newPortID";
Debug.Print("Set port ID = " + cPointData.PortID);

Debug.Print("Get CPoint name = " + cPointData.Name2);
cPointData.Name2 = "myCPoint";
Debug.Print("Set CPoint name = " + cPointData.Name2);

vLocation = cPointData.Location;

double[] location;
location = (double[])vLocation;
xLoc = location[0];
yLoc = location[1];
zLoc = location[2];

Debug.Print("Get location: " + xLoc + " " + yLoc + " " + zLoc);

vDirection = cPointData.Direction;

double[] direction;
direction = (double[])vDirection;
xDir = direction[0];
yDir = direction[1];
zDir = direction[2];

Debug.Print("Get direction vector: " + xDir + " " + yDir + " " + zDir);
}
Part.ForceRebuild3(false);
cPointData = null;
featData = null;
}

public SldWorks swApp;

}
}

```

Get and Set Connection Point Feature Data Example (VBA)

This example shows how to get and set data for connection point features.

```
'-----
' Preconditions:
' 1. Ensure that the latest SOLIDWORKS Design Library is loaded.
' 2. Drag and drop design library > routing > electrical > db9 male.sldprt
'    into the main view.
' 3. Observe CPoint1 in the FeatureManager design tree.
' 4. Run (F5) this macro.

' Postconditions:
' 1. Inspect the Immediate Window.
' 2. CPoint1 is now MyCPoint in the FeatureManager design tree.
'
' NOTE: Because this is a Design Library part, close it without saving changes.
'-----
```

```
Dim swApp As SOLIDWORKS.Application
Option Explicit
Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim Part As SldWorks.ModelDoc2
    Dim selMgr As SldWorks.SelectionMgr

    Dim selCount As Long
    Dim selType As Long
    Dim selObj As Object
    Dim selFeat As SldWorks.Feature

    Dim vLocation As Variant
    Dim xLoc As Double, yLoc As Double, zLoc As Double

    Dim vDirection As Variant
    Dim xDir As Double, yDir As Double, zDir As Double

    Dim boolstatus As Boolean

    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc
    Set selMgr = Part.SelectionManager

    boolstatus = Part.Extension.SelectByID2("CPoint1", "CONNECTIONPOINT", 0, 0, 0, False, 0, Nothing, 0)
    selCount = selMgr.GetSelectedObjectCount2(-1)
    If (selCount > 0) Then
        selType = selMgr.GetSelectedObjectType3(1, -1)
        Set selObj = selMgr.GetSelectedObject6(1, -1)
        If (selType = SwConst.swSelCONNECTIONPOINTS) Then
            Set selFeat = selObj
        End If
    End If

    Part.ClearSelection2 True

    Dim featData As Object
    Set featData = selFeat.GetDefinition()

    Dim cPointData As SldWorks.ConnectionPointFeatureData
    Set cPointData = featData

    If Not cPointData Is Nothing Then
        Debug.Print "Get stublength = " & cPointData.StubLength
        cPointData.StubLength = 0.009
        Debug.Print "Set stublength = " & cPointData.StubLength

        Debug.Print "Get diameter = " & cPointData.RouteDiameter
        cPointData.RouteDiameter = 0.011
        Debug.Print "Set diameter = " & cPointData.RouteDiameter

        Debug.Print "Get route type = " & cPointData.RouteType
        cPointData.RouteType = 2
        Debug.Print "Set route type = " & cPointData.RouteType

        Debug.Print "Get route sub type = " & cPointData.RouteSubType
        cPointData.RouteSubType = 3
        Debug.Print "Set route sub type = " & cPointData.RouteSubType

        Debug.Print "Get electrical Pin ID = " & cPointData.ElectricalPinID
        cPointData.ElectricalPinID = "zxc"
        Debug.Print "Set electrical Pin ID = " & cPointData.ElectricalPinID

        Debug.Print "Get port ID = " & cPointData.PortID
        cPointData.PortID = "newPortID"
        Debug.Print "Set port ID = " & cPointData.PortID

        Debug.Print "Get CPoint name = " & cPointData.Name2
        cPointData.Name2 = "myCPoint"
```

```
Debug.Print "Set CPoint name = " & cPointData.Name2

vLocation = cPointData.Location
xLoc = vLocation(0)
yLoc = vLocation(1)
zLoc = vLocation(2)
Debug.Print "Get location: " & xLoc & " " & yLoc & " " & zLoc

vDirection = cPointData.Direction
xDir = vDirection(0)
yDir = vDirection(1)
zDir = vDirection(2)
Debug.Print "Get direction vector: " & xDir & " " & yDir & " " & zDir

End If
Part.ForceRebuild3(false)
Set cPointData = Nothing
Set featData = Nothing
End Sub
```

Get and Set Connection Point Feature Data Example (VB.NET)

This example shows how to get and set data for connection point features.

```

'-----
' Preconditions:
' 1. Ensure that the latest SOLIDWORKS Design Library is loaded.
' 2. Drag and drop design library > routing > electrical > db9 male.sldprt
'    into the main view.
' 3. Observe CPoint1 in the FeatureManager design tree.
' 4. Run (F5) this macro.
'
' Postconditions:
' 1. Inspect the Immediate Window.
' 2. CPoint1 is now MyCPoint in the FeatureManager design tree.
'
' NOTE: Because this is a Design Library part, close it without saving changes.
'-----

```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Imports System
```

```
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim Part As ModelDoc2
```

```
        Dim selMgr As SelectionMgr
```

```
        Dim selCount As Long
```

```
        Dim selType As Long
```

```
        Dim selObj As Object
```

```
        Dim selFeat As Feature = Nothing
```

```
        Dim vLocation As Object
```

```
        Dim xLoc As Double, yLoc As Double, zLoc As Double
```

```
        Dim vDirection As Object
```

```
        Dim xDir As Double, yDir As Double, zDir As Double
```

```
        Dim boolstatus As Boolean
```

```
        Part = swApp.ActiveDoc
```

```
        selMgr = Part.SelectionManager
```

```
        boolstatus = Part.Extension.SelectByID2("CPoint1", "CONNECTIONPOINT", 0, 0, 0, False, 0, Nothing, 0)
```

```
        selCount = selMgr.GetSelectedObjectCount2(-1)
```

```
        If (selCount > 0) Then
```

```
            selType = selMgr.GetSelectedObjectType3(1, -1)
```

```
            selObj = selMgr.GetSelectedObject6(1, -1)
```

```
            If (selType = swSelectType_e.swSelCONNECTIONPOINTS) Then
```

```
                selFeat = selObj
```

```
            End If
```

```
        End If
```

```
        Part.ClearSelection2(True)
```

```
        Dim featData As Object = Nothing
```

```
        If Not IsNothing(selFeat) Then
```

```
            featData = selFeat.GetDefinition()
```

```
        End If
```

```
        Dim cPointData As IConnectionPointFeatureData
```

```
        cPointData = featData
```

```
        If Not cPointData Is Nothing Then
```

```
            Debug.Print("Get stublength = " & cPointData.StubLength)
```

```
            cPointData.StubLength = 0.009
```

```
            Debug.Print("Set stublength = " & cPointData.StubLength)
```

```
            Debug.Print("Get diameter = " & cPointData.RouteDiameter)
```

```
            cPointData.RouteDiameter = 0.011
```

```
            Debug.Print("Set diameter = " & cPointData.RouteDiameter)
```

```
            Debug.Print("Get route type = " & cPointData.RouteType)
```

```
            cPointData.RouteType = 2
```

```
            Debug.Print("Set route type = " & cPointData.RouteType)
```

```
            Debug.Print("Get route sub type = " & cPointData.RouteSubType)
```

```
            cPointData.RouteSubType = 3
```

```
            Debug.Print("Set route sub type = " & cPointData.RouteSubType)
```

```
            Debug.Print("Get electrical Pin ID = " & cPointData.ElectricalPinID)
```

```
            cPointData.ElectricalPinID = "zxc"
```

```
            Debug.Print("Set electrical Pin ID = " & cPointData.ElectricalPinID)
```

```
            Debug.Print("Get port ID = " & cPointData.PortID)
```

```
            cPointData.PortID = "newPortID"
```

```
            Debug.Print("Set port ID = " & cPointData.PortID)
```

```

Debug.Print("Get CPoint name = " & cPointData.Name2)
cPointData.Name2 = "myCPoint"
Debug.Print("Set CPoint name = " & cPointData.Name2)

vLocation = cPointData.Location
xLoc = vLocation(0)
yLoc = vLocation(1)
zLoc = vLocation(2)
Debug.Print("Get location: " & xLoc & " " & yLoc & " " & zLoc)

vDirection = cPointData.Direction
xDir = vDirection(0)
yDir = vDirection(1)
zDir = vDirection(2)
Debug.Print("Get direction vector: " & xDir & " " & yDir & " " & zDir)

End If
Part.ForceRebuild3(false)
cPointData = Nothing
featData = Nothing
End Sub

```

```

Public swApp As SldWorks

```

```

End Class

```

Get and Set Constraint for Dome Feature Example (VBA)

This example shows how to get and set a constraining point for a dome feature. A part containing a dome feature constrained by a sketch point on the origin is open.

```
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.PartDoc  
Dim component As SldWorks.Component2  
Dim newPointFeat As SldWorks.SketchPoint  
Dim dome As SldWorks.feature  
Dim domeConstraintPoint As SldWorks.SketchPoint  
Dim dome_featData As SldWorks.DomeFeatureData2  
Dim boolstatus As Variant  
Sub main()  
{  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    boolstatus = Part.Extension.SelectByID2("Point1@Sketch1", "EXTSKETCHPOINT", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)  
    Set newPointFeat = Part.SelectionManager.GetSelectedObject5(1)  
  
    Set dome = Part.FeatureByName("Dome1")  
    Set dome_featData = dome.GetDefinition  
    boolstatus = dome_featData.AccessSelections(Part, component)  
    Set domeConstraintPoint = dome_featData.ConstraintPointOrSketch  
  
    If Not domeConstraintPoint Is Nothing Then  
  
        dome_featData.ConstraintPointOrSketch = newPointFeat  
  
        boolstatus = dome.ModifyDefinition(dome_featData, Part, Nothing)  
  
    End If  
  
    dome_featData.ReleaseSelectionAccess  
  
}  
End Sub
```

Get and Set Dimension Extension Lines Gaps (VBA)

This example shows how to get and set the gaps for the extension lines of a display dimension.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\replaceview.slddrw.
' 2. Select a vertical edge in the model in DrawingView1.
'
' Postconditions:
' 1. Modifies the selected edge's dimension's line
'    extension gaps to the specified value.
' 2. Examine the drawing.
'
' NOTE: Because this drawing is used elsewhere, do not save
'       changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDisplayDim As SldWorks.DisplayDimension
retBool As Boolean
Dim UseDoc As Boolean
Dim Gap As Double

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Programmatically add a dimension to the selected edge
    Set swDisplayDim = swModel.AddDimension2(0.07991695284535, 0.1237274983129, 0)

    Debug.Print "Dimension Extension Lines"
    Debug.Print ""

    ' Get document default value for extension lines' gaps
    Debug.Print "Value of document default gap: " & swModel.GetUserPreferenceDoubleValue(swDetailingWitnessLineGap)
    Debug.Print " "

    ' Get a dimension's extension line's gap
    retBool = swDisplayDim.GetWitnessLineGap(1, UseDoc, Gap)
    Debug.Print "Get Gap Operation:"
    Debug.Print "  Use document default for gap: " & UseDoc
    Debug.Print "  Gap value in system units:      " & Gap
    Debug.Print " "

    ' Set the dimension's extension line's gaps
    Gap = 0.005
    retBool = swDisplayDim.SetWitnessLineGap(0, False, Gap)
    retBool = swDisplayDim.SetWitnessLineGap(1, False, Gap)
    Debug.Print "Set Gap Operation: "
    Debug.Print "  Gap value in system units:      " & Gap
    Debug.Print " "

    ' Get the dimension's extension gap again
    retBool = swDisplayDim.GetWitnessLineGap(1, UseDoc, Gap)
    Debug.Print "Get Gap Operation:"
    Debug.Print "  Use document default for gap: " & UseDoc
    Debug.Print "  Gap value in system units:      " & Gap
    Debug.Print " "

    swModel.GraphicsRedraw

End Sub
```

Get and Set Direction for Dome Feature Example (VBA)

This example shows how to get and set the direction of a dome feature. You must have a part containing a dome feature created with an edge indicating the direction of the feature and preselect that edge.

```
'-----  
Option Explicit  
Dim swApp As SldWorks.SldWorks  
Dim part As SldWorks.PartDoc  
Dim component As SldWorks.Component2  
Dim newEdge As SldWorks.Edge  
Dim dome As SldWorks.feature  
Dim dome_featData As SldWorks.DomeFeatureData2  
Dim domeDirection As SldWorks.Edge  
Dim boolstatus As Variant  
Sub main()  
{  
    Set swApp = Application.SldWorks  
    Set part = swApp.ActiveDoc  
    Set newEdge = part.SelectionManager.GetSelectedObject5(1)  
    Set dome = part.FeatureByName("Dome1")  
    Set dome_featData = dome.GetDefinition  
    boolstatus = dome_featData.AccessSelections(part, component)  
  
    Set domeDirection = dome_featData.Direction  
  
    If Not domeDirection Is Nothing Then  
  
        dome_featData.Direction = newEdge  
  
        boolstatus = dome.ModifyDefinition(dome_featData, part, Nothing)  
  
    End If  
    dome_featData.ReleaseSelectionAccess  
}  
End Sub
```

Get and Set FeatureManager Design Tree Display (C#)

This example shows how to get and set properties related to displaying the FeatureManager design tree.

```
//-----
// Preconditions:
// 1. Verify that the specified document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the assembly and gets the FeatureManager design tree's display-related property
//    design tree's display-related property values.
// 2. Examine both the Immediate window and the FeatureManager
//    design tree, then press F5.
// 3. Re-examine both the Immediate window and the FeatureManager
//    design tree to verify the changes.
//
// NOTE: Because this assembly document is used
// elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace ViewFeatureManagerDesignTreeCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModelDoc = default(ModelDoc2);
            FeatureManager swFeatMgr = default(FeatureManager);
            string document = null;
            int errors = 0;
            int warnings = 0;

            document = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\bladed
shaft.sldasm";

            swModelDoc = (ModelDoc2)swApp.OpenDoc6(document, (int)swDocumentTypes_e.swDocASSEMBLY,
(int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swFeatMgr = (FeatureManager)swModelDoc.FeatureManager;

            Debug.Print("----- Before changing FeatureManager design tree properties -----");
            Debug.Print("View dependencies      = " + swFeatMgr.ViewDependencies);
            Debug.Print("View features          = " + swFeatMgr.ViewFeatures);
            Debug.Print("Show feature details    = " + swFeatMgr.ShowFeatureDetails);
            Debug.Print("Show hierarchy only     = " + swFeatMgr.ShowHierarchyOnly);

            System.Diagnostics.Debugger.Break();
            // Examine the Immediate window and
            // FeatureManager design tree before
            // resuming the macro

            // Change details, dependencies, hierarchy, and
            // features-related properties
            if ((swFeatMgr.ViewDependencies))
            {
                swFeatMgr.ViewFeatures = true;
            }
            else
            {
                swFeatMgr.ViewDependencies = true;
            }

            if ((swFeatMgr.ShowFeatureDetails))
            {
                swFeatMgr.ShowHierarchyOnly = true;
            }
            else
            {
                swFeatMgr.ShowFeatureDetails = true;
            }

            Debug.Print("----- After changing FeatureManager design tree properties -----");
            Debug.Print("View dependencies      = " + swFeatMgr.ViewDependencies);
            Debug.Print("View features          = " + swFeatMgr.ViewFeatures);
            Debug.Print("Show feature details    = " + swFeatMgr.ShowFeatureDetails);
            Debug.Print("Show hierarchy only     = " + swFeatMgr.ShowHierarchyOnly);
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>

    public SldWorks swApp;
```


Get and Set FeatureManager Design Tree Display (VBA)

This example shows how to get and set properties related to displaying the FeatureManager design tree.

```
'-----
' Preconditions:
' 1. Verify that the specified document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly and gets the FeatureManager
'    design tree's display-related property values.
' 2. Examine both the Immediate window and the FeatureManager
'    design tree, then press F5.
' 3. Re-examine both the Immediate window the
'    FeatureManager design tree to verify the changes.
'
' NOTE: Because this assembly document is used
' elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Dim swFeatMgr As SldWorks.FeatureManager
Dim document As String
Dim errors As Long
Dim warnings As Long

Sub main()

document = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\bladed shaft.sldasm"

Set swApp = Application.SldWorks
Set swModelDoc = swApp.OpenDoc6(document, swDocASSEMBLY, swOpenDocOptions_Silent, "", errors, warnings)
Set swFeatMgr = swModelDoc.FeatureManager

Debug.Print "----- Before changing FeatureManager design tree properties -----"
Debug.Print "View dependencies      = " & swFeatMgr.ViewDependencies
Debug.Print "View features                = " & swFeatMgr.ViewFeatures
Debug.Print "Show feature details          = " & swFeatMgr.ShowFeatureDetails
Debug.Print "Show hierarchy only           = " & swFeatMgr.ShowHierarchyOnly

Stop
' Examine the Immediate window and
' FeatureManager design tree before
' resuming the macro

' Change details, dependencies, hierarchy, and
' features-related properties
If (swFeatMgr.ViewDependencies) Then
    swFeatMgr.ViewFeatures = True
Else
    swFeatMgr.ViewDependencies = True
End If

If (swFeatMgr.ShowFeatureDetails) Then
    swFeatMgr.ShowHierarchyOnly = True
Else
    swFeatMgr.ShowFeatureDetails = True
End If

Debug.Print "----- After changing FeatureManager design tree properties -----"
Debug.Print "View dependencies      = " & swFeatMgr.ViewDependencies
Debug.Print "View features                = " & swFeatMgr.ViewFeatures
Debug.Print "Show feature details          = " & swFeatMgr.ShowFeatureDetails
Debug.Print "Show hierarchy only           = " & swFeatMgr.ShowHierarchyOnly

End Sub
```

Get and Set FeatureManager Design Tree Display (VB.NET)

This example shows how to get and set properties related to displaying the FeatureManager design tree.

```

'-----
' Preconditions:
' 1. Verify that the specified document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the assembly and gets the FeatureManager
'    design tree's display-related property values.
' 2. Examine both the Immediate window and the FeatureManager
'    design tree, then press F5.
' 3. Re-examine both the Immediate window the
'    FeatureManager design tree to verify the changes.
'
' NOTE: Because this assembly document is used
' elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub Main()

        Dim swModelDoc As ModelDoc2
        Dim swFeatMgr As FeatureManager
        Dim document As String
        Dim errors As Integer
        Dim warnings As Integer

        document = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\bladed shaft.sldasm"

        swModelDoc = swApp.OpenDoc6(document, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "",
errors, warnings)
        swFeatMgr = swModelDoc.FeatureManager

        Debug.Print("----- Before changing FeatureManager design tree properties -----")
        Debug.Print("View dependencies      = " & swFeatMgr.ViewDependencies)
        Debug.Print("View features          = " & swFeatMgr.ViewFeatures)
        Debug.Print("Show feature details    = " & swFeatMgr.ShowFeatureDetails)
        Debug.Print("Show hierarchy only     = " & swFeatMgr.ShowHierarchyOnly)

        Stop
        ' Examine the Immediate window and
        ' FeatureManager design tree before
        ' resuming the macro

        ' Change details, dependencies, hierarchy, and
        ' features-related properties
        If (swFeatMgr.ViewDependencies) Then
            swFeatMgr.ViewFeatures = True
        Else
            swFeatMgr.ViewDependencies = True
        End If

        If (swFeatMgr.ShowFeatureDetails) Then
            swFeatMgr.ShowHierarchyOnly = True
        Else
            swFeatMgr.ShowFeatureDetails = True
        End If

        Debug.Print("----- After changing FeatureManager design tree properties -----")
        Debug.Print("View dependencies      = " & swFeatMgr.ViewDependencies)
        Debug.Print("View features          = " & swFeatMgr.ViewFeatures)
        Debug.Print("Show feature details    = " & swFeatMgr.ShowFeatureDetails)
        Debug.Print("Show hierarchy only     = " & swFeatMgr.ShowHierarchyOnly)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Get and Set File Summary Information Example (VBA)

This example shows how to get and set the file summary information in a SOLIDWORKS Design document.

```
'-----
' Preconditions:
' 1. Verify that the specified part template exists.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Opens a new part document.
' 2. Sets summary information for the part document.
' 3. Examine the Immediate window.
' 4. Click File > Properties and examine the Summary
'    Information dialog.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim Text As String

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2015\templates\Part.prtdot", 0, 0, 0)

    ' Set and get the Title KeyWord in the File Summary Info
    swModel.SummaryInfo(swSumInfoTitle) = "TitleXX"
    Text = swModel.SummaryInfo(swSumInfoTitle)

    ' Set and get the Subject KeyWord in the File Summary Info
    swModel.SummaryInfo(swSumInfoSubject) = "SubjectXX"
    Text = swModel.SummaryInfo(swSumInfoTitle)

    ' Set and get the Author KeyWord in the File Summary Info
    swModel.SummaryInfo(swSumInfoAuthor) = "AuthorXX"
    Text = swModel.SummaryInfo(swSumInfoAuthor)

    ' Set and get the Keywords KeyWord in the File Summary Info
    swModel.SummaryInfo(swSumInfoKeywords) = "KeywordsXX"
    Text = swModel.SummaryInfo(swSumInfoKeywords)

    ' Set and get the Comment KeyWord in the File Summary Info
    swModel.SummaryInfo(swSumInfoComment) = "CommentXX"
    Text = swModel.SummaryInfo(swSumInfoComment)

    ' Get the read-only Summary info fields
    Text = swModel.SummaryInfo(swSumInfoSavedBy)
    Debug.Print "Last saved by: " & Text
    Text = swModel.SummaryInfo(swSumInfoCreateDate)
    Debug.Print "Created: " & Text
    Text = swModel.SummaryInfo(swSumInfoSaveDate)
    Debug.Print "Last saved: " & Text

End Sub
```

Get and Set Format of Note Example (VBA)

This example shows how to get and set the format of a selected note.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\2012-sm.slddrw.
' 2. Select the Fixed face note.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Reformats the selected note.
' 2. Click the drawing and examine the note.
' 3. Examine the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swNote As SldWorks.Note
Dim swSelMgr As SldWorks.SelectionMgr

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    ' Get selected rich-text formatted note
    Set swNote = swSelMgr.GetSelectedObject6(1, 0)

    ' Print the current note including the formatting tags
    Debug.Print "Before reformatting: " & swNote.PropertyLinkedText

    ' Change the color of the selected note to red and bold
    ' all of the text
    swNote.PropertyLinkedText = "<FONT color=0x000000ff><FONT style=B>This is a test of formatting a note using the API"

    ' Print the current note including the formatting tags
    Debug.Print "After reformatting: " & swNote.PropertyLinkedText

    ' Rebuild the document to see the newly formatted note
    swModel.EditRebuild3

End Sub
```

Get and Set Hole Callout Variables Example (C#)

This example shows how to get and set hole callout variables.

```
//-----
// Preconditions:
// 1. Open a drawing document containing a hole callout.
// 2. Select the hole callout.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Iterates through the variables in the hole callout and
//    gets any angle, length, and string callout
//    variables.
// 2. If the hole callout variable is a length variable,
//    then sets its precision and tolerance type.
// 3. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            DisplayDimension swDisplayDimension = default(DisplayDimension);
            CalloutLengthVariable swCalloutLengthVariable = default(CalloutLengthVariable);
            CalloutAngleVariable swCalloutAngleVariable = default(CalloutAngleVariable);
            CalloutStringVariable swCalloutStringVariable = default(CalloutStringVariable);
            CalloutVariable swCalloutVariable = default(CalloutVariable);
            int i = 0;
            int vType = 0;
            object[] holeVariables = null;
            string str1 = null;
            string str2 = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            //Get the selected hole callout
            swDisplayDimension = (DisplayDimension)swSelMgr.GetSelectedObject6(1, -1);
            holeVariables = (object[])swDisplayDimension.GetHoleCalloutVariables();
            Debug.Print("Number of hole callout variables = " + holeVariables.Length);
            Debug.Print("");

            //Determine type of hole callout variable and get and set some values
            for (i = 0; i < holeVariables.Length; i++)
            {
                swCalloutVariable = (CalloutVariable)holeVariables[i];
                str1 = " Callout variable name = " + swCalloutVariable.VariableName;
                str2 = " Callout variable name as it appears in Dimension PropertyManager page = " + swCalloutVariable.UserReadableVariableName;
                vType = swCalloutVariable.Type;
                if (vType == (int)swCalloutVariableType_e.swCalloutVariableType_Length)
                {
                    swCalloutLengthVariable = (CalloutLengthVariable)swCalloutVariable;
                    Debug.Print("Callout variable(" + i + ")'s" + " type = length");
                    Debug.Print(str1);
                    Debug.Print(str2);
                    Debug.Print(" Length = " + swCalloutLengthVariable.Length);
                    Debug.Print(" Precision = " + swCalloutLengthVariable.Precision);
                    Debug.Print(" Tolerance precision = " + swCalloutLengthVariable.TolerancePrecision);
                    swCalloutLengthVariable.Precision = swCalloutLengthVariable.Precision - 1 - i;
                    Debug.Print(" Changed precision = " + swCalloutLengthVariable.Precision);
                    swCalloutVariable.ToleranceType = (int)swTolType_e.swTolBILAT;
                }
                else if (vType == (int)swCalloutVariableType_e.swCalloutVariableType_Angle)
                {
                    swCalloutAngleVariable = (CalloutAngleVariable)swCalloutVariable;
                    Debug.Print("Callout variable(" + i + ")'s" + " type = angle");
                    Debug.Print(str1);
                    Debug.Print(str2);
                    Debug.Print(" Angle = " + swCalloutAngleVariable.Angle);
                }
                else if (vType == (int)swCalloutVariableType_e.swCalloutVariableType_String)
                {
                    swCalloutStringVariable = (CalloutStringVariable)swCalloutVariable;
                    Debug.Print("Callout variable(" + i + ")'s" + " type = string");
                    Debug.Print(str1);
                    Debug.Print(str2);
                    Debug.Print(" String = '" + swCalloutStringVariable.String + "'");
                }
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
```

Get and Set Hole Callout Variables Example (VBA)

This example shows how to get and set hole callout variables.

```
'-----
' Preconditions:
' 1. Open a drawing document containing a hole callout.
' 2. Select the hole callout.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Iterates through the variables in the hole callout and
'    gets any angle, length, and string callout
'    variables.
' 2. If the hole callout variable is a length variable,
'    then sets its precision and tolerance type.
' 3. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swDisplayDimension As SldWorks.DisplayDimension
Dim swCalloutLengthVariable As SldWorks.CalloutLengthVariable
Dim swCalloutAngleVariable As SldWorks.CalloutAngleVariable
Dim swCalloutStringVariable As SldWorks.CalloutStringVariable
Dim swCalloutVariable As SldWorks.CalloutVariable
Dim i As Long
Dim vType As Long
Dim holeVariables As Variant
Dim str1 As String
Dim str2 As String

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    'Get the selected hole callout
    Set swDisplayDimension = swSelMgr.GetSelectedObject6(1, -1)
    holeVariables = swDisplayDimension.GetHoleCalloutVariables
    Debug.Print "Number of hole callout variables = " & UBound(holeVariables) + 1
    Debug.Print ""

    'Determine type of hole callout variable and get and set some values
    For i = 0 To UBound(holeVariables)
        Set swCalloutVariable = holeVariables(i)
        str1 = "    Callout variable name = " & swCalloutVariable.VariableName
        str2 = "    Callout variable name as it appears in Dimension PropertyManager page = " & swCalloutVariable.UserReadableVariableName
        vType = swCalloutVariable.Type
        If vType = swCalloutVariableType_e.swCalloutVariableType_Length Then
            Set swCalloutLengthVariable = swCalloutVariable
            Debug.Print "Callout variable(" & i & ")'"s" & " type = length"
            Debug.Print str1
            Debug.Print str2
            Debug.Print "    Length = " & swCalloutLengthVariable.Length
            Debug.Print "    Precision = " & swCalloutLengthVariable.precision
            Debug.Print "    Tolerance precision = " & swCalloutLengthVariable.TolerancePrecision
            swCalloutLengthVariable.precision = swCalloutLengthVariable.precision - 1 - i
            Debug.Print "    Changed precision = " & swCalloutLengthVariable.precision
            swCalloutVariable.ToleranceType = swTolType_e.swTolBILAT
        ElseIf vType = swCalloutVariableType_e.swCalloutVariableType_Angle Then
            Set swCalloutAngleVariable = swCalloutVariable
            Debug.Print "Callout variable(" & i & ")'"s" & " type = angle"
            Debug.Print str1
            Debug.Print str2
            Debug.Print "    Angle = " & swCalloutAngleVariable.Angle
        ElseIf vType = swCalloutVariableType_e.swCalloutVariableType_String Then
            Set swCalloutStringVariable = swCalloutVariable
            Debug.Print "Callout variable(" & i & ")'"s" & " type = string"
            Debug.Print str1
            Debug.Print str2
            Debug.Print "    String = '" & swCalloutStringVariable.String & "'"
        End If
    Next
End Sub
```

Get and Set Hole Callout Variables Example (VB.NET)

This example shows how to get and set hole callout variables.

```

-----
Preconditions:
1. Open a drawing document containing a hole callout.
2. Select the hole callout.
3. Open the Immediate window.
Postconditions:
1. Iterates through the variables in the hole callout and
   gets any angle, length, and string callout
   variables.
2. If the hole callout variable is a length variable,
   then sets its precision and tolerance type.
3. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro
    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swDisplayDimension As DisplayDimension
        Dim swCalloutLengthVariable As CalloutLengthVariable
        Dim swCalloutAngleVariable As CalloutAngleVariable
        Dim swCalloutStringVariable As CalloutStringVariable
        Dim swCalloutVariable As CalloutVariable
        Dim i As Integer
        Dim vType As Integer
        Dim holeVariables As Object
        Dim str1 As String
        Dim str2 As String

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager

        'Get the selected hole callout
        swDisplayDimension = swSelMgr.GetSelectedObject6(1, -1)
        holeVariables = swDisplayDimension.GetHoleCalloutVariables
        Debug.Print("Number of hole callout variables = " & UBound(holeVariables) + 1)
        Debug.Print("")

        'Determine type of hole callout variable and get and set some values
        For i = 0 To UBound(holeVariables)
            swCalloutVariable = holeVariables(i)
            str1 = " Callout variable name = " & swCalloutVariable.VariableName
            str2 = " Callout variable name as it appears in Dimension PropertyManager page = " & swCalloutVariable.UserReadableVariableName
            vType = swCalloutVariable.Type
            If vType = swCalloutVariableType.e.swCalloutVariableType_Length Then
                swCalloutLengthVariable = swCalloutVariable
                Debug.Print("Callout variable(" & i & ")'s" & " type = length")
                Debug.Print(str1)
                Debug.Print(str2)
                Debug.Print(" Length = " & swCalloutLengthVariable.Length)
                Debug.Print(" Precision = " & swCalloutLengthVariable.Precision)
                Debug.Print(" Tolerance precision = " & swCalloutLengthVariable.TolerancePrecision)
                swCalloutLengthVariable.Precision = swCalloutLengthVariable.Precision - 1 - i
                Debug.Print(" Changed precision = " & swCalloutLengthVariable.Precision)
                swCalloutVariable.ToleranceType = swTolType.e.swTolBILAT
            ElseIf vType = swCalloutVariableType.e.swCalloutVariableType_Angle Then
                swCalloutAngleVariable = swCalloutVariable
                Debug.Print("Callout variable(" & i & ")'s" & " type = angle")
                Debug.Print(str1)
                Debug.Print(str2)
                Debug.Print(" Angle = " & swCalloutAngleVariable.Angle)
            ElseIf vType = swCalloutVariableType.e.swCalloutVariableType_String Then
                swCalloutStringVariable = swCalloutVariable
                Debug.Print("Callout variable(" & i & ")'s" & " type = string")
                Debug.Print(str1)
                Debug.Print(str2)
                Debug.Print(" String = '" & swCalloutStringVariable.String & "'")
            End If
        Next

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks
End Class

```

Get and Set Large Design Review Marks for Configurations Example (C#)

This example shows how to get and set the Large Design Review marks for all of the configurations in an assembly.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\driveworksexpress\mobile gantry.sldasm.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets the current Large Design Review marks for all configurations.
// 2. Sets the Large Design Review marks for all configurations to true.
// 3. Gets the modified Large Design Review marks for all configurations.
// 4. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Configuration swConf = default(Configuration);
            ConfigurationManager swConfMgr = default(ConfigurationManager);
            string[] configNameArr = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swConfMgr = (ConfigurationManager)swModel.ConfigurationManager;
            configNameArr = (string[])swModel.GetConfigurationNames();

            //Get current Large Design Review marks for all configurations
            Debug.Print("Current Large Design Review marks for configurations:");
            foreach (string configName in configNameArr)
            {
                swConf = (Configuration)swModel.GetConfigurationByName(configName);
                Debug.Print(" " + configName + ": " + swConf.LargeDesignReviewMark);
            }

            //Set Large Design Review marks for all configurations to true
            foreach (string configName in configNameArr)
            {
                swConf = (Configuration)swModel.GetConfigurationByName(configName);
                swConf.LargeDesignReviewMark = true;
            }

            //Get modified Large Design Review marks for all configurations
            Debug.Print("Modified Large Design Review marks for configurations:");
            foreach (string configName in configNameArr)
            {
                swConf = (Configuration)swModel.GetConfigurationByName(configName);
                Debug.Print(" " + configName + ": " + swConf.LargeDesignReviewMark);
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get and Set Large Design Review Marks for Configurations Example (VBA)

This example shows how to get and set the Large Design Review marks for all of the configurations in an assembly.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\driveworksxpess\mobile gantry.sldasm.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets the current Large Design Review marks for all configurations.
' 2. Sets the Large Design Review marks for all configurations to true.
' 3. Gets the modified Large Design Review marks for all configurations.
' 4. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
```

```
Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swConf As SldWorks.Configuration
    Dim swConfMgr As SldWorks.ConfigurationManager
    Dim configNameArr As Variant
    Dim configName As Variant

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swConfMgr = swModel.ConfigurationManager
    configNameArr = swModel.GetConfigurationNames

    'Get current Large Design Review marks for all configurations
    Debug.Print "Current Large Design Review marks for configurations:"
    For Each configName In configNameArr
        Set swConf = swModel.GetConfigurationByName(configName)
        Debug.Print " " & configName & ": " & swConf.LargeDesignReviewMark
    Next

    'Set Large Design Review marks for all configurations to true
    For Each configName In configNameArr
        Set swConf = swModel.GetConfigurationByName(configName)
        swConf.LargeDesignReviewMark = True
    Next

    'Get modified Large Design Review marks for all configurations
    Debug.Print "Modified Large Design Review marks for configurations:"
    For Each configName In configNameArr
        Set swConf = swModel.GetConfigurationByName(configName)
        Debug.Print " " & configName & ": " & swConf.LargeDesignReviewMark
    Next

End Sub
```

Get and Set Large Design Review Marks for Configurations Example (VB.NET)

This example shows how to get and set the Large Design Review marks for all of the configurations in an assembly.

```

-----
Preconditions:
1. Open public_documents\samples\tutorial\driveworksexpress\mobile gantry.sldasm.
2. Open the Immediate window.

Postconditions:
1. Gets the current Large Design Review marks for all configurations.
2. Sets the Large Design Review marks for all configurations to true.
3. Gets the modified Large Design Review marks for all configurations.
4. Examine the Immediate window.

NOTE: Because the assembly is used elsewhere, do not save changes.
-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swConf As Configuration
        Dim swConfMgr As ConfigurationManager
        Dim configNameArr As Object
        Dim configName As Object

        swModel = swApp.ActiveDoc
        swConfMgr = swModel.ConfigurationManager
        configNameArr = swModel.GetConfigurationNames

        'Get current Large Design Review marks for all configurations
        Debug.Print("Current Large Design Review marks for configurations:")
        For Each configName In configNameArr
            swConf = swModel.GetConfigurationByName(configName)
            Debug.Print(" " & configName & ": " & swConf.LargeDesignReviewMark)
        Next

        'Set Large Design Review marks for all configurations to true
        For Each configName In configNameArr
            swConf = swModel.GetConfigurationByName(configName)
            swConf.LargeDesignReviewMark = True
        Next

        'Get modified Large Design Review marks for all configurations
        Debug.Print("Modified Large Design Review marks for configurations:")
        For Each configName In configNameArr
            swConf = swModel.GetConfigurationByName(configName)
            Debug.Print(" " & configName & ": " & swConf.LargeDesignReviewMark)
        Next

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

```

```
End Class
```

Get and Set Material Density Example (VBA)

This example shows how to get and set the material density for a part.

```
'-----  
' Preconditions:  
' 1. Open a part.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Gets and sets the part's density.  
' 2. Examine the Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim nDensity As Double  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    Debug.Print "File   = " + swModel.GetPathName  
  
    ' Density is returned in metric units, i.e., kg/m^3  
    nDensity = swModel.GetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swMaterialPropertyDensity)  
    Debug.Print "  Get current density = " & nDensity & " kg/m^3"  
  
    swModel.SetUserPreferenceDoubleValue swMaterialPropertyDensity, nDensity / 2#  
    nDensity = swModel.GetUserPreferenceDoubleValue(swUserPreferenceDoubleValue_e.swMaterialPropertyDensity)  
    Debug.Print "  Set new density = " & nDensity & " kg/m^3"  
  
End Sub
```

Get and Set Material Visual Properties Example (C#)

This method shows how to get and set the material properties of a part.

```
//-----
// Preconditions:
// 1. Open a part.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Gets and sets the part's material visual properties.
// 2. Inspect the Immediate window and graphics area.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace ApplyAppearance_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 myModel = default(ModelDoc2);
            PartDoc myPart = default(PartDoc);
            MaterialVisualPropertiesData myMatVisProps = default(MaterialVisualPropertiesData);
            string configName = null;
            string databaseName = null;
            string newPropName = null;
            bool orgBlend = false;
            bool orgApply = false;
            double orgAngle = 0;
            double orgScale = 0;
            long longstatus = 0;

            myModel = (ModelDoc2)swApp.ActiveDoc;
            myPart = (PartDoc)myModel;
            myMatVisProps = myPart.GetMaterialVisualProperties();

            Debug.Print("==== Material Visual Properties Example =====");

            if ((myMatVisProps != null))
            {
                dump_material_visual_properties(myMatVisProps, myPart);

                // Set the material to something else, so that the display changes
                configName = "default";
                databaseName = "SOLIDWORKS Materials";
                newPropName = "Beech";
                myPart.SetMaterialPropertyName2(configName, databaseName, newPropName);
                dump_material_visual_properties(myMatVisProps, myPart);
            }

            // Set the material visual properties to be just color, no advanced graphics
            myMatVisProps = myPart.GetMaterialVisualProperties();

            if ((myMatVisProps != null))
            {
                longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);
                dump_material_visual_properties(myMatVisProps, myPart);

                // Set the material visual properties to be RealView
                myMatVisProps.RealView = true;
                longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);
                dump_material_visual_properties(myMatVisProps, myPart);

                // Set the material visual properties to be SOLIDWORKS standard textures
                myMatVisProps.RealView = false;
                longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);
                dump_material_visual_properties(myMatVisProps, myPart);
            }

            myMatVisProps = myPart.GetMaterialVisualProperties();

            if ((myMatVisProps != null))
            {
                orgAngle = myMatVisProps.Angle;
                myMatVisProps.Angle = orgAngle + 1.0;
                longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);
            }
        }
    }
}
```

```

dump_material_visual_properties(myMatVisProps, myPart);
orgScale = myMatVisProps.Scale2;
myMatVisProps.Scale2 = orgScale * 1.25;
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);

dump_material_visual_properties(myMatVisProps, myPart);

// Toggle the standard texture to blend with the part color
if (myMatVisProps.BlendColor == false)
{
    orgBlend = false;
}
else
{
    orgBlend = true;
}

myMatVisProps.BlendColor = !orgBlend;
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);

dump_material_visual_properties(myMatVisProps, myPart);
myMatVisProps.BlendColor = orgBlend;
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);

dump_material_visual_properties(myMatVisProps, myPart);

// Toggle the apply material color to part flag
if (myMatVisProps.ApplyMaterialColorToPart == false)
{
    orgApply = false;
}
else
{
    orgApply = true;
}

myMatVisProps.ApplyMaterialColorToPart = !orgApply;
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);

dump_material_visual_properties(myMatVisProps, myPart);
myMatVisProps.ApplyMaterialColorToPart = orgApply;
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);

dump_material_visual_properties(myMatVisProps, myPart);

// Toggle the apply material hatch to drawing section view flag
if (myMatVisProps.ApplyMaterialHatchToSection == false)
{
    orgApply = false;
}
else
{
    orgApply = true;
}

myMatVisProps.ApplyMaterialHatchToSection = !orgApply;
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);

dump_material_visual_properties(myMatVisProps, myPart);
myMatVisProps.ApplyMaterialHatchToSection = orgApply;
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);

dump_material_visual_properties(myMatVisProps, myPart);

// Toggle the apply appearance flag
if (myMatVisProps.ApplyAppearance == false)
{
    orgApply = false;
}
else
{
    orgApply = true;
}

myMatVisProps.ApplyAppearance = !orgApply;
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);

dump_material_visual_properties(myMatVisProps, myPart);
myMatVisProps.ApplyAppearance = orgApply;
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, (int)swInConfigurationOpts_e.swThisConfiguration,
null);

dump_material_visual_properties(myMatVisProps, myPart);
}

}

private void dump_material_visual_properties(MaterialVisualPropertiesData myMatVisProps, PartDoc myPart)
{
    string configName = null;
    string databaseName = null;
    string propName = null;
    bool bRealView = false;
    double dScale = 0;
    double dAngle = 0;
    bool bBlendColor = false;
    bool bApplyColor = false;

```

```

bool bApplyHatch = false;
bool bApplyAppearance = false;
configName = "default";
databaseName = null;
propName = myPart.GetMaterialPropertyName2(configName, out databaseName);
Debug.Print("");
Debug.Print("Config: \"" + configName + "\", Database: \"" + databaseName + "\", Material: \"" + propName + "\"");

```

```

if ((myMatVisProps != null))

```

```

{
    bRealView = myMatVisProps.RealView;
    dScale = myMatVisProps.Scale2;
    dAngle = myMatVisProps.Angle;
    bBlendColor = myMatVisProps.BlendColor;
    bApplyColor = myMatVisProps.ApplyMaterialColorToPart;
    bApplyHatch = myMatVisProps.ApplyMaterialHatchToSection;
    bApplyAppearance = myMatVisProps.ApplyAppearance;

```

```

    if (bRealView == false)
    
```

```

    {
        Debug.Print("Advanced graphics - SOLIDWORKS standard textures.");
    }

```

```

    else
    
```

```

    {
        Debug.Print("Advanced graphics - RealView textures.");
    }

```

```

    Debug.Print("    SOLIDWORKS standard texture scale = " + dScale + ", Angle = " + dAngle);

```

```

    if (bBlendColor == false)
    
```

```

    {
        Debug.Print("    Do not blend part color with SOLIDWORKS standard texture.");
    }

```

```

    else
    
```

```

    {
        Debug.Print("    Blend part color with SOLIDWORKS standard texture.");
    }

```

```

    if (bApplyColor == false)
    
```

```

    {
        Debug.Print("Do not apply material color to part.");
    }

```

```

    else
    
```

```

    {
        Debug.Print("Apply material color to part.");
    }

```

```

    if (bApplyHatch == false)
    
```

```

    {
        Debug.Print("Do not apply material hatch to drawing section.");
    }

```

```

    else
    
```

```

    {
        Debug.Print("Apply material hatch to drawing section.");
    }

```

```

    if (bApplyAppearance == false)
    
```

```

    {
        Debug.Print("Do not apply appearance.");
    }

```

```

    else
    
```

```

    {
        Debug.Print("Apply appearance.");
    }

```

```

}

```

```

}

```

```

public SldWorks swApp;

```

```

}

```

```

}

```

Get and Set Material Visual Properties Example (VBA)

This method shows how to get and set the material properties of a part.

```
'-----
' Preconditions:
' 1. Open a part.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets and sets the part's material visual properties.
' 2. Inspect the Immediate window and graphics area.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim myModel As SldWorks.ModelDoc2
    Dim myPart As SldWorks.PartDoc
    Dim myMatVisProps As SldWorks.MaterialVisualPropertiesData
    Dim configName As String, databaseName As String
    Dim newPropName As String
    Dim orgBlend As Boolean, orgApply As Boolean
    Dim orgAngle As Double
    Dim orgScale As Double
    Dim longstatus As Long

    Set swApp = Application.SldWorks
    Set myModel = swApp.ActiveDoc
    Set myPart = myModel
    Set myMatVisProps = myPart.GetMaterialVisualProperties()

    Debug.Print "===== Material Visual Properties Example ====="

    If Not myMatVisProps Is Nothing Then
        Call dump_material_visual_properties(myMatVisProps, myPart)

        ' Set the material to something else, so that the display changes
        configName = "default"
        databaseName = "SOLIDWORKS Materials"
        newPropName = "Beech"
        myPart.SetMaterialPropertyName2 configName, databaseName, newPropName
        Call dump_material_visual_properties(myMatVisProps, myPart)
    End If

    ' Set the material visual properties to be just color, no advanced graphics
    Set myMatVisProps = myPart.GetMaterialVisualProperties()

    If Not myMatVisProps Is Nothing Then
        longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
        Call dump_material_visual_properties(myMatVisProps, myPart)

        ' Set the material visual properties to be RealView
        myMatVisProps.RealView = True
        longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
        Call dump_material_visual_properties(myMatVisProps, myPart)

        ' Set the material visual properties to be SOLIDWORKS standard textures
        myMatVisProps.RealView = False
        longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
        Call dump_material_visual_properties(myMatVisProps, myPart)
    End If

    Set myMatVisProps = myPart.GetMaterialVisualProperties()

    If Not myMatVisProps Is Nothing Then
        orgAngle = myMatVisProps.Angle
        myMatVisProps.Angle = orgAngle + 1#
        longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
        Call dump_material_visual_properties(myMatVisProps, myPart)
        orgScale = myMatVisProps.Scale2
        myMatVisProps.Scale2 = orgScale * 1.25
        longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
        Call dump_material_visual_properties(myMatVisProps, myPart)

        ' Toggle the standard texture to blend with the part color
        If myMatVisProps.BlendColor = 0 Then
            orgBlend = False
        Else
            orgBlend = True
        End If

        myMatVisProps.BlendColor = Not orgBlend
        longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
        Call dump_material_visual_properties(myMatVisProps, myPart)
        myMatVisProps.BlendColor = orgBlend
        longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
        Call dump_material_visual_properties(myMatVisProps, myPart)

        ' Toggle the apply material color to part flag
```

```

    If myMatVisProps.ApplyMaterialColorToPart = 0 Then
        orgApply = False
    Else
        orgApply = True
    End If

    myMatVisProps.ApplyMaterialColorToPart = Not orgApply
    longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
    Call dump_material_visual_properties(myMatVisProps, myPart)
    myMatVisProps.ApplyMaterialColorToPart = orgApply
    longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
    Call dump_material_visual_properties(myMatVisProps, myPart)

    ' Toggle the apply material hatch to drawing section view flag

    If myMatVisProps.ApplyMaterialHatchToSection = 0 Then
        orgApply = False
    Else
        orgApply = True
    End If

    myMatVisProps.ApplyMaterialHatchToSection = Not orgApply
    longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
    Call dump_material_visual_properties(myMatVisProps, myPart)
    myMatVisProps.ApplyMaterialHatchToSection = orgApply
    longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
    Call dump_material_visual_properties(myMatVisProps, myPart)

    ' Toggle the apply appearance flag
    If myMatVisProps.ApplyAppearance = 0 Then
        orgApply = False
    Else
        orgApply = True
    End If

    myMatVisProps.ApplyAppearance = Not orgApply
    longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
    Call dump_material_visual_properties(myMatVisProps, myPart)
    myMatVisProps.ApplyAppearance = orgApply
    longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swThisConfiguration, Nothing)
    Call dump_material_visual_properties(myMatVisProps, myPart)
End If
End Sub

Private Sub dump_material_visual_properties(myMatVisProps As SldWorks.MaterialVisualPropertiesData, myPart As SldWorks.PartDoc)
    Dim configName As String, databaseName As String
    Dim propName As String
    Dim bRealView As Boolean
    Dim dScale As Double, dAngle As Double
    Dim bBlendColor As Boolean, bApplyColor As Boolean, bApplyHatch As Boolean, bApplyAppearance As Boolean
    configName = "default"
    propName = myPart.GetMaterialPropertyName2(configName, databaseName)
    Debug.Print ""
    Debug.Print "Config: "" & configName & """, Database: "" & databaseName & """, Material: "" & propName & """"

    If Not myMatVisProps Is Nothing Then
        bRealView = myMatVisProps.RealView
        dScale = myMatVisProps.Scale2
        dAngle = myMatVisProps.Angle
        bBlendColor = myMatVisProps.BlendColor
        bApplyColor = myMatVisProps.ApplyMaterialColorToPart
        bApplyHatch = myMatVisProps.ApplyMaterialHatchToSection
        bApplyAppearance = myMatVisProps.ApplyAppearance

        If bRealView = 0 Then
            Debug.Print "Advanced graphics - SOLIDWORKS standard textures."
        Else
            Debug.Print "Advanced graphics - RealView textures."
        End If
        Debug.Print "    SOLIDWORKS standard texture scale = " & dScale & ", Angle = " & dAngle
        If bBlendColor = 0 Then
            Debug.Print "    Do not blend part color with SOLIDWORKS standard texture."
        Else
            Debug.Print "    Blend part color with SOLIDWORKS standard texture."
        End If

        If bApplyColor = 0 Then
            Debug.Print "Do not apply material color to part."
        Else
            Debug.Print "Apply material color to part."
        End If

        If bApplyHatch = 0 Then
            Debug.Print "Do not apply material hatch to drawing section."
        Else
            Debug.Print "Apply material hatch to drawing section."
        End If

        If bApplyAppearance = 0 Then
            Debug.Print "Do not apply appearance."
        Else
            Debug.Print "Apply appearance."
        End If
    End If
End Sub

```

Get and Set Material Visual Properties Example (VB.NET)

This method shows how to get and set the material properties of a part.

```

'-----
' Preconditions:
' 1. Open a part.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Gets and sets the part's material visual properties.
' 2. Inspect the Immediate window and graphics area.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Sub main()

        Dim myModel As ModelDoc2
        Dim myPart As PartDoc
        Dim myMatVisProps As MaterialVisualPropertiesData
        Dim configName As String, databaseName As String
        Dim newPropName As String
        Dim orgBlend As Boolean, orgApply As Boolean
        Dim orgAngle As Double
        Dim orgScale As Double
        Dim longstatus As Long

        myModel = swApp.ActiveDoc
        myPart = myModel
        myMatVisProps = myPart.GetMaterialVisualProperties()

        Debug.Print("==== Material Visual Properties Example =====")

        If Not myMatVisProps Is Nothing Then
            Call dump_material_visual_properties(myMatVisProps, myPart)

            ' Set the material to something else, so that the display changes
            configName = "default"
            databaseName = "SOLIDWORKS Materials"
            newPropName = "Beech"
            myPart.SetMaterialPropertyName2(configName, databaseName, newPropName)
            Call dump_material_visual_properties(myMatVisProps, myPart)
        End If

        ' Set the material visual properties to be just color, no advanced graphics
        myMatVisProps = myPart.GetMaterialVisualProperties()

        If Not myMatVisProps Is Nothing Then
            longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
            Call dump_material_visual_properties(myMatVisProps, myPart)

            ' Set the material visual properties to be RealView
            myMatVisProps.RealView = True
            longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
            Call dump_material_visual_properties(myMatVisProps, myPart)

            ' Set the material visual properties to be SOLIDWORKS standard textures
            myMatVisProps.RealView = False
            longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
            Call dump_material_visual_properties(myMatVisProps, myPart)
        End If

        myMatVisProps = myPart.GetMaterialVisualProperties()

        If Not myMatVisProps Is Nothing Then
            orgAngle = myMatVisProps.Angle
            myMatVisProps.Angle = orgAngle + 1.0#
            longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
            Call dump_material_visual_properties(myMatVisProps, myPart)
            orgScale = myMatVisProps.Scale2
            myMatVisProps.Scale2 = orgScale * 1.25
            longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
            Call dump_material_visual_properties(myMatVisProps, myPart)

            ' Toggle the standard texture to blend with the part color
            If myMatVisProps.BlendColor = 0 Then
                orgBlend = False
            Else
                orgBlend = True
            End If

            myMatVisProps.BlendColor = Not orgBlend
            longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
        End If
    End Sub
End Class

```

```

Call dump_material_visual_properties(myMatVisProps, myPart)
myMatVisProps.BlendColor = orgBlend
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
Call dump_material_visual_properties(myMatVisProps, myPart)

' Toggle the apply material color to part flag
If myMatVisProps.ApplyMaterialColorToPart = 0 Then
    orgApply = False
Else
    orgApply = True
End If

myMatVisProps.ApplyMaterialColorToPart = Not orgApply
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
Call dump_material_visual_properties(myMatVisProps, myPart)
myMatVisProps.ApplyMaterialColorToPart = orgApply
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
Call dump_material_visual_properties(myMatVisProps, myPart)

' Toggle the apply material hatch to drawing section view flag
If myMatVisProps.ApplyMaterialHatchToSection = 0 Then
    orgApply = False
Else
    orgApply = True
End If

myMatVisProps.ApplyMaterialHatchToSection = Not orgApply
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
Call dump_material_visual_properties(myMatVisProps, myPart)
myMatVisProps.ApplyMaterialHatchToSection = orgApply
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
Call dump_material_visual_properties(myMatVisProps, myPart)

' Toggle the apply appearance flag
If myMatVisProps.ApplyAppearance = 0 Then
    orgApply = False
Else
    orgApply = True
End If

myMatVisProps.ApplyAppearance = Not orgApply
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
Call dump_material_visual_properties(myMatVisProps, myPart)
myMatVisProps.ApplyAppearance = orgApply
longstatus = myPart.SetMaterialVisualProperties(myMatVisProps, swInConfigurationOpts_e.swThisConfiguration, Nothing)
Call dump_material_visual_properties(myMatVisProps, myPart)
End If

End Sub

Private Sub dump_material_visual_properties(ByVal myMatVisProps As MaterialVisualPropertiesData, ByVal myPart As PartDoc)

    Dim configName As String, databaseName As String
    Dim propName As String
    Dim bRealView As Boolean
    Dim dScale As Double, dAngle As Double
    Dim bBlendColor As Boolean, bApplyColor As Boolean, bApplyHatch As Boolean, bApplyAppearance As Boolean
    configName = "default"
    databaseName = Nothing
    propName = myPart.GetMaterialPropertyName2(configName, databaseName)
    Debug.Print("")
    Debug.Print("Config: "" & configName & """, Database: "" & databaseName & "", Material: "" & propName & "")

    If Not myMatVisProps Is Nothing Then
        bRealView = myMatVisProps.RealView
        dScale = myMatVisProps.Scale2
        dAngle = myMatVisProps.Angle
        bBlendColor = myMatVisProps.BlendColor
        bApplyColor = myMatVisProps.ApplyMaterialColorToPart
        bApplyHatch = myMatVisProps.ApplyMaterialHatchToSection
        bApplyAppearance = myMatVisProps.ApplyAppearance

        If bRealView = 0 Then
            Debug.Print("Advanced graphics - SOLIDWORKS standard textures.")
        Else
            Debug.Print("Advanced graphics - RealView textures.")
        End If
        Debug.Print("    SOLIDWORKS standard texture scale = " & dScale & ", Angle = " & dAngle)
        If bBlendColor = 0 Then
            Debug.Print("    Do not blend part color with SOLIDWORKS standard texture.")
        Else
            Debug.Print("    Blend part color with SOLIDWORKS standard texture.")
        End If

        If bApplyColor = 0 Then
            Debug.Print("Do not apply material color to part.")
        Else
            Debug.Print("Apply material color to part.")
        End If

        If bApplyHatch = 0 Then
            Debug.Print("Do not apply material hatch to drawing section.")
        Else
            Debug.Print("Apply material hatch to drawing section.")
        End If
    End If
End Sub

```

```
End If
If bApplyAppearance = 0 Then
    Debug.Print("Do not apply appearance.")
Else
    Debug.Print("Apply appearance.")
End If

End If

End Sub

Public swApp As SldWorks

End Class
```

Get and Set Model Break View Display Example (C#)

This example shows how to get and set whether to display an assembly's Model Break View in a drawing.

```
//-----
// Preconditions:
// 1. Open a drawing that has:
//    * assembly with Model Break View1.
//    * Drawing View1.
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Displays Model Break View1.
// 2. Inspect the Immediate window and graphics area.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace ModelBreakView_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        View swView;
        SelectionMgr swSelMgr;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)Part.SelectionManager;

            boolstatus = ((DrawingDoc)Part).ActivateView("Drawing View1");
            boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0.395146689428525, 0.746414551874767, 0, false,
0, null, 0);

            swView = (View)swSelMgr.GetSelectedObject6(1, -1);
            boolstatus = swView.ShowModelBreakState(true, "Model Break View1");

            Debug.Print("Displaying model break view? " + swView.IsModelBreakState());

            Part.ForceRebuild3(true);
        }

        public SldWorks swApp;
    }
}
```

Get and Set Model Break View Display Example (VBA)

This example shows how to get and set whether to display an assembly's Model Break View in a drawing.

```
'-----  
' Preconditions:  
' 1. Open a drawing that has:  
'    * assembly with Model Break View1.  
'    * Drawing View1.  
' 2. Open an Immediate window.  
'  
' Postconditions:  
' 1. Displays Model Break View1.  
' 2. Inspect the Immediate window and graphics area.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim swView As SldWorks.View  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim boolstatus As Boolean  
  
Option Explicit  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    Set swSelMgr = Part.SelectionManager  
  
    boolstatus = Part.ActivateView("Drawing View1")  
    boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0.395146689428525, 0.746414551874767, 0, False, 0,  
Nothing, 0)  
  
    Set swView = swSelMgr.GetSelectedObject6(1, -1)  
    boolstatus = swView.ShowModelBreakState(True, "Model Break View1")  
  
    Debug.Print "Displaying Model Break View? " & swView.IsModelBreakState  
    Part.ForceRebuild3 True  
  
End Sub
```

Get and Set Model Break View Display Example (VB.NET)

This example shows how to get and set whether to display an assembly's Model Break View in a drawing.

```

'-----
' Preconditions:
' 1. Open a drawing that has:
'    * assembly with Model Break View1.
'    * Drawing View1.
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Displays Model Break View1.
' 2. Inspect the Immediate window and graphics area.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim Part As ModelDoc2
    Dim swView As View
    Dim swSelMgr As SelectionMgr
    Dim boolstatus As Boolean

```

```
    Sub main()
```

```

        Part = swApp.ActiveDoc
        swSelMgr = Part.SelectionManager

```

```

        boolstatus = Part.ActivateView("Drawing View1")
        boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0.395146689428525, 0.746414551874767, 0, False, 0,
Nothing, 0)

```

```

        swView = swSelMgr.GetSelectedObject6(1, -1)
        boolstatus = swView.ShowModelBreakState(True, "Model Break View1")

```

```
        Debug.Print("Displaying model break view? " & swView.IsModelBreakState)
```

```
        Part.ForceRebuild3(True)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get and Set Model View HLR Quality Example (C#)

This example shows how to get and set the hidden-line removal quality of model edges.

```
//-----
// Preconditions: Open a part or assembly document.
//
// Postconditions:
// 1. Sets the hidden-line removal quality to
//    swHlrQuality_e.swFastHlr for a fast redraw and lower
//    image quality of model edges.
// 2. Examine the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace HLRQuality_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            {
                ModelDoc2 swModel = default(ModelDoc2);
                ModelView swModView = default(ModelView);

                swModel = (ModelDoc2)swApp.ActiveDoc;
                swModView = (ModelView)swModel.ActiveView;

                // Set HLR quality to a fast redraw and lower image quality of model edges
                swModView.HlrQuality = (int)swHlrQuality_e.swFastHlr;

                Debug.Print("File = " + swModel.GetPathName());
                Debug.Print(" Hidden-line removal quality as defined in swHlrQuality_e = " + swModView.HlrQuality);
            }

            public SldWorks swApp;
        }
    }
}
```

Get and Set Model View HLR Quality Example (VBA)

This example shows how to get and set the hidden-line removal quality of model edges.

```
'-----  
' Preconditions: Open a part or assembly document.  
'  
' Postconditions:  
' 1. Sets the hidden-line removal quality to  
'    swHlrQuality_e.swFastHlr for a fast redraw and lower  
'    image quality of model edges.  
' 2. Examine the Immediate window.  
'-----
```

Option Explicit

```
Sub main()  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swModView            As SldWorks.ModelView  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModView = swModel.ActiveView  
  
    ' Set HLR quality to a fast redraw and lower image quality of model edges  
    swModView.HlrQuality = swFastHlr  
  
    Debug.Print "File = " & swModel.GetPathName  
    Debug.Print " Hidden-line removal quality as defined in swHlrQuality_e (1 = swHlrQuality_e.swFastHlr)= " &  
    swModView.HlrQuality  
  
End Sub
```

Get and Set Model View HLR Quality Example (VB.NET)

This example shows how to get and set the hidden-line removal quality of model edges.

```
'-----
' Preconditions: Open a part or assembly document.
'
' Postconditions:
' 1. Sets the hidden-line removal quality to
'    swHlrQuality_e.swFastHlr for a fast redraw and lower
'    image quality of model edges.
' 2. Examine the Immediate window.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim swModView As ModelView
```

```
        swModel = swApp.ActiveDoc
        swModView = swModel.ActiveView
```

```
        ' Set HLR quality to a fast redraw and lower image quality of model edges
        swModView.HlrQuality = swHlrQuality_e.swFastHlr
```

```
        Debug.Print("File = " & swModel.GetPathName)
```

```
        Debug.Print(" Hidden-line removal quality as defined in swHlrQuality_e (1 = swHlrQuality_e.swFastHlr) = " &
        swModView.HlrQuality)
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Get and Set Name of Note Example (VBA)

This example shows how to get the name of the selected note and change its name.

```
'-----  
' Precondition:  
' 1. Open public_documents\samples\tutorial\api\2012-sm.sldrw.  
' 2. Select the Fixed face note.  
'  
' Postcondition:  
' 1. Changes the name of the selected note to SM Fixed face.  
' 2. Click OK to close each message box.  
'  
' NOTE: Because the drawing is used elsewhere, do not save changes.  
'-----  
Option Explicit
```

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swDrawing As SldWorks.DrawingDoc  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swNote As SldWorks.Note  
Dim swAnn As SldWorks.Annotation  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swNote = swSelMgr.GetSelectedObject6(1, 0)  
  
    ' Get note annotation  
    Set swAnn = swNote.GetAnnotation  
    ' Get annotation name  
    MsgBox swAnn.GetName  
    ' Change annotation name; new name must be unique  
    swAnn.SetName "SM Fixed face"  
    MsgBox swAnn.GetName  
  
End Sub
```

Get and Set Paragraph Properties Example (C#)

This example shows how to get and set the properties of paragraphs in note annotations.

```
//-----
// Preconditions:
// 1. Open a drawing with a note annotation that contains one or more
//    paragraphs.
// 2. Select the annotation in the graphics area.
// 3. Open an Immediate window.
//
// Postconditions:
// 1. For each paragraph in the selected note:
//    * Sets the paragraph and line spacing.
//    * Gets properties of numbered lists, if present.
//    * Gets text segment formatting.
//    * Bold the text of each text segment.
// 2. Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetParagraphs_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Note noteObj = default(Note);
            Annotation swAnn = default(Annotation);
            Paragraphs paragraphs = default(Paragraphs);
            int numSegs = 0;
            int paragraphIndex = 0;
            int iseg = 0;
            TextFormat textFormat = default(TextFormat);
            int paraType = 0;
            int numberingtype = 0;
            int startas = 0;
            int format = 0;
            int numtype = 0;
            int numParagraphs = 0;
            string Text = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            if (swSelMgr.GetSelectedObjectCount() > 0)
            {
                noteObj = (Note)swSelMgr.GetSelectedObject6(1, -1);
                swAnn = (Annotation)noteObj.GetAnnotation();
                paragraphs = (Paragraphs)swAnn.GetParagraphs();

                numParagraphs = paragraphs.Count;
                Debug.Print("Number of paragraphs = " + numParagraphs);
                Debug.Print("");

                for (paragraphIndex = 0; paragraphIndex <= numParagraphs - 1; paragraphIndex++)
                {
                    paragraphs.CurrentParagraph = paragraphIndex;
                    Debug.Print("paragraph(" + paragraphIndex + "): ");
                    Debug.Print(paragraphs.GetText(true));

                    if (paragraphIndex == 0)
                    {
                        paragraphs.SetFormatting(0.0011, 0.0012);
                    }
                    else
                    {
                        paragraphs.SetFormatting(0.0021, 0.0022);
                    }

                    paragraphs.GetBulletsAndNumbering(out paraType, out numberingtype, out startas, out numtype, out format);
                    Debug.Print("Paragraph list type as defined in swParagraphType_e: " + paraType);
                    if (paraType == 1)
                    {
                        Debug.Print("Numbered list:");
                        Debug.Print(" Start numbering from location as defined in swNumberedListStartType_e: " + numberingtype);
                    }
                }
            }
        }
    }
}
```

```

        Debug.Print(" Start numbering index: " + startas);
        Debug.Print(" Type as defined in swNumberedListType_e: " + numtype);
        Debug.Print(" Format as defined in swNumberingFormat_e: " + format);
    }

    numSegs = paragraphs.GetTextSegmentCount();
    Debug.Print("Text segment count: " + numSegs);

    for (iseg = 0; iseg <= numSegs - 1; iseg++)
    {
        Text = paragraphs.GetTextSegmentText(iseg);
        Debug.Print("segment(" + iseg + "): " + Text);

        textFormat = (TextFormat)paragraphs.GetTextSegmentFormat(iseg);
        Debug.Print(" Typeface: " + textFormat.TypeFaceName);
        Debug.Print(" Backwards? " + textFormat.BackWards);
        Debug.Print(" Bold? " + textFormat.Bold);
        Debug.Print(" Italic? " + textFormat.Italic);
        Debug.Print(" Strikeout? " + textFormat.Strikeout);

        textFormat.Bold = true;
        paragraphs.SetTextSegmentFormat(iseg, textFormat);
    }

    paragraphs.UpdateParagraph();
}

swModel.GraphicsRedraw2();
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

public SldWorks swApp;
}

}

```

Get and Set Paragraph Properties Example (VBA)

This example shows how to get and set the properties of paragraphs in note annotations.

```

'-----
' Preconditions:
' 1. Open a drawing with a note annotation that contains one or more
'    paragraphs.
' 2. Select the annotation in the graphics area.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. For each paragraph in the selected note:
'    * Sets the paragraph and line spacing.
'    * Gets properties of numbered lists, if present.
'    * Gets text segment formatting.
'    * Bolds the text of each text segment.
' 2. Inspect the Immediate window.
'-----

```

Option Explicit

```

Sub main()
    Dim swApp           As SldWorks.SldWorks
    Dim swModel          As SldWorks.ModelDoc2
    Dim swSelMgr          As SldWorks.SelectionMgr
    Dim swAnnObj          As Object
    Dim swAnn             As SldWorks.Annotation
    Dim paragraphs        As SldWorks.paragraphs
    Dim numSegs           As Long
    Dim paragraphIndex    As Long
    Dim iseg              As Long
    Dim textFormat        As SldWorks.textFormat
    Dim paraType          As Long
    Dim numberingtype     As Long
    Dim startas           As Long
    Dim format            As Long
    Dim numtype           As Long
    Dim numParagraphs     As Long
    Dim Text              As String

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager

    If swSelMgr.GetSelectedObjectCount > 0 Then

        Set swAnnObj = swSelMgr.GetSelectedObject6(1, -1)
        Set swAnn = swAnnObj.GetAnnotation
        Set paragraphs = swAnn.GetParagraphs

        numParagraphs = paragraphs.count
        Debug.Print "Number of paragraphs = " & numParagraphs
        Debug.Print ""

        For paragraphIndex = 0 To numParagraphs - 1

            paragraphs.CurrentParagraph = paragraphIndex
            Debug.Print "paragraph(" & paragraphIndex & "): "; paragraphs.GetText(True)

            If paragraphIndex = 0 Then
                paragraphs.SetFormatting 0.0011, 0.0012
            Else
                paragraphs.SetFormatting 0.0021, 0.0022
            End If

            paragraphs.GetBulletsAndNumbering paraType, numberingtype, startas, numtype, format
            Debug.Print "Paragraph list type as defined in swParagraphType_e: " & paraType
            If paraType = 1 Then
                Debug.Print "Numbered list:"
                Debug.Print "  Start numbering from location as defined in swNumberedListStartType_e: " & numberingtype
                Debug.Print "  Start numbering index: " & startas
                Debug.Print "  Type as defined in swNumberedListType_e: " & numtype
                Debug.Print "  Format as defined in swNumberingFormat_e: " & format
            End If

            numSegs = paragraphs.GetTextSegmentCount
            Debug.Print "Text segment count: " & numSegs

            For iseg = 0 To numSegs - 1
                Text = paragraphs.GetTextSegmentText(iseg)
                Debug.Print "segment(" & iseg & "): " & Text
            Next iseg
        Next paragraphIndex
    End If
End Sub

```

```
Set textFormat = paragraphs.GetTextSegmentFormat(iseg)
Debug.Print " Typeface: " & textFormat.TypeFaceName
Debug.Print " Backwards? " & textFormat.BackWards
Debug.Print " Bold? " & textFormat.Bold
Debug.Print " Italic? " & textFormat.Italic
Debug.Print " Strikeout? " & textFormat.Strikeout
```

```
textFormat.Bold = True
paragraphs.SetTextSegmentFormat iseg, textFormat
```

```
Next
```

```
paragraphs.UpdateParagraph
```

```
Next
```

```
swModel.GraphicsRedraw2
```

```
End If
```

```
End Sub
```

Get and Set Paragraph Properties Example (VB.NET)

This example shows how to get and set the properties of paragraphs in note annotations.

```

'-----
' Preconditions:
' 1. Open a drawing with a note annotation that contains one or more
'    paragraphs.
' 2. Select the annotation in the graphics area.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. For each paragraph in the selected note:
'    * Sets the paragraph and line spacing.
'    * Gets properties of numbered lists, if present.
'    * Gets text segment formatting.
'    * Bolds the text of each text segment.
' 2. Inspect the Immediate window.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swAnnObj As Note
        Dim swAnn As Annotation
        Dim paragraphs As Paragraphs
        Dim numSegs As Integer
        Dim paragraphIndex As Integer
        Dim iseg As Integer
        Dim textFormat As TextFormat
        Dim paraType As Integer
        Dim numberingtype As Integer
        Dim startas As Integer
        Dim format As Integer
        Dim numtype As Integer
        Dim numParagraphs As Integer
        Dim Text As String
    
```

```

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
    
```

```
    If swSelMgr.GetSelectedObjectCount > 0 Then
```

```

        swAnnObj = swSelMgr.GetSelectedObject6(1, -1)
        swAnn = swAnnObj.GetAnnotation()
        paragraphs = swAnn.GetParagraphs

        numParagraphs = paragraphs.count
        Debug.Print("Number of paragraphs = " & numParagraphs)
        Debug.Print("")
    
```

```
    For paragraphIndex = 0 To numParagraphs - 1
```

```

        paragraphs.CurrentParagraph = paragraphIndex
        Debug.Print("paragraph(" & paragraphIndex & "): ")
        Debug.Print(paragraphs.GetText(True))
    
```

```

        If paragraphIndex = 0 Then
            paragraphs.SetFormatting(0.0011, 0.0012)
        Else
            paragraphs.SetFormatting(0.0021, 0.0022)
        End If
    
```

```

        paragraphs.GetBulletsAndNumbering(paraType, numberingtype, startas, numtype, format)
        Debug.Print("Paragraph list type as defined in swParagraphType_e: " & paraType)
    
```

```

        If paraType = 1 Then
            Debug.Print("Numbered list:")
            Debug.Print("  Start numbering from location as defined in swNumberedListStartType_e: " & numberingtype)
            Debug.Print("  Start numbering index: " & startas)
            Debug.Print("  Type as defined in swNumberedListType_e: " & numtype)
            Debug.Print("  Format as defined in swNumberingFormat_e: " & format)
        End If
    
```

```

        numSegs = paragraphs.GetTextSegmentCount
        Debug.Print("Text segment count: " & numSegs)
    
```

```

        For iseg = 0 To numSegs - 1
            Text = paragraphs.GetTextSegmentText(iseg)
            Debug.Print("segment(" & iseg & "): " & Text)
        Next iseg
    End Sub
End Class

```

```
textFormat = paragraphs.GetTextSegmentFormat(iseg)
Debug.Print("  Typeface: " & textFormat.TypeFaceName)
Debug.Print("  Backwards? " & textFormat.BackWards)
Debug.Print("  Bold? " & textFormat.Bold)
Debug.Print("  Italic? " & textFormat.Italic)
Debug.Print("  Strikeout? " & textFormat.Strikeout)

textFormat.Bold = True
paragraphs.SetTextSegmentFormat(iseg, textFormat)
```

```
Next
```

```
paragraphs.UpdateParagraph()
```

```
Next
```

```
swModel.GraphicsRedraw2()
```

```
End If
```

```
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Get and Set Parting Line Feature Data Example (C#)

This example shows how to get and set parting line feature data.

```
//-----
// Preconditions:
// 1. Open a model document that contains a parting line feature.
// 2. Select the parting line feature in the FeatureManager design tree.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Gets and sets parting line feature data.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace PartingLineFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            PartingLineFeatureData swPartingLineFeatureData = default(PartingLineFeatureData);
            bool state = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Get parting line feature
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swPartingLineFeatureData = (PartingLineFeatureData)swFeature.GetDefinition();

            //Roll back to get and set parting line feature data
            state = swPartingLineFeatureData.AccessSelections(swModel, null);
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" Feature name: " + swFeature.Name);
            Debug.Print(" Parting line status as defined by swPartingLineFeatureData.Status: " + swPartingLineFeatureData.Status());
            Debug.Print(" Draft angle: " + swPartingLineFeatureData.Angle);
            //Reverse pull direction of part line
            Debug.Print(" Pull direction before reversing: " + swPartingLineFeatureData.PullDirection);
            if (swPartingLineFeatureData.PullDirection)
            {
                swPartingLineFeatureData.PullDirection = false;
            }
            else
            {
                swPartingLineFeatureData.PullDirection = true;
            }
            Debug.Print(" Pull direction after reversing: " + swPartingLineFeatureData.PullDirection);
            Debug.Print(" Pull direction type as defined by swSelectType_e: " + swPartingLineFeatureData.PullDirectionType);
            Debug.Print(" Split faces: " + swPartingLineFeatureData.SplitFaces);
            if (swPartingLineFeatureData.SplitFaces)
            {
                Debug.Print(" Split faces options as defined by swSplitFacesOption_e: " + swPartingLineFeatureData.SplitFacesOption);
            }
            Debug.Print(" Core/cavity split: " + swPartingLineFeatureData.CoreCavitySplit);
            Debug.Print(" Number of entities to split: " + swPartingLineFeatureData.GetEntitiesToSplitCount());

            //Modify and roll forward parting line feature
            swFeature.ModifyDefinition(swPartingLineFeatureData, swModel, null);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get and Set Parting Line Feature Data Example (VBA)

This example shows how to get and set parting line feature data.

```
'-----
' Preconditions:
' 1. Open a model document that contains a parting line feature.
' 2. Select the parting line feature in the FeatureManager design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets and sets parting line feature data.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swPartingLineFeatureData As SldWorks.PartingLineFeatureData
Dim state As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    'Get parting line feature
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swPartingLineFeatureData = swFeature.GetDefinition

    'Roll back to get and set parting line feature data
    state = swPartingLineFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Feature name: " & swFeature.Name
    Debug.Print " Parting line status as defined by swPartingLineFeatureData.status_e: " & swPartingLineFeatureData.status
    Debug.Print " Draft angle: " & swPartingLineFeatureData.Angle

    'Reverse pull direction of part line
    Debug.Print " Pull direction before reversing: " & swPartingLineFeatureData.PullDirection
    If swPartingLineFeatureData.PullDirection Then
        swPartingLineFeatureData.PullDirection = False
    Else
        swPartingLineFeatureData.PullDirection = True
    End If
    Debug.Print " Pull direction after reversing: " & swPartingLineFeatureData.PullDirection

    Debug.Print " Pull direction type as defined by swSelectType_e: " & swPartingLineFeatureData.PullDirectionType
    Debug.Print " Split faces: " & swPartingLineFeatureData.SplitFaces
    If swPartingLineFeatureData.SplitFaces Then
        Debug.Print " Split faces options as defined by swSplitFacesOption_e: " & swPartingLineFeatureData.SplitFacesOption
    End If
    Debug.Print " Core/cavity split: " & swPartingLineFeatureData.CoreCavitySplit
    Debug.Print " Number of entities to split: " & swPartingLineFeatureData.GetEntitiesToSplitCount

    'Modify and roll forward parting line feature
    swFeature.ModifyDefinition swPartingLineFeatureData, swModel, Nothing

End Sub
```

Get and Set Parting Line Feature Data Example (VB.NET)

This example shows how to get and set parting line feature data.

```

-----
Preconditions:
1. Open a model document that contains a parting line feature.
2. Select the parting line feature in the FeatureManager design tree.
3. Open the Immediate window.
-----
Postconditions:
1. Gets and sets parting line feature data.
2. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swPartingLineFeatureData As PartingLineFeatureData
        Dim state As Boolean

        swModel = swApp.ActiveDoc

        'Get parting line feature
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swPartingLineFeatureData = swFeature.GetDefinition

        'Roll back to get and set parting line feature data
        state = swPartingLineFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" Feature name: " & swFeature.Name)
        Debug.Print(" Parting line status as defined by swPartingLineFeatureData.Status_e: " & swPartingLineFeatureData.Status)
        Debug.Print(" Draft angle: " & swPartingLineFeatureData.Angle)
        'Reverse pull direction of part line
        Debug.Print(" Pull direction before reversing: " & swPartingLineFeatureData.PullDirection)
        If swPartingLineFeatureData.PullDirection Then
            swPartingLineFeatureData.PullDirection = False
        Else
            swPartingLineFeatureData.PullDirection = True
        End If
        Debug.Print(" Pull direction after reversing: " & swPartingLineFeatureData.PullDirection)
        Debug.Print(" Pull direction type as defined by swSelectType_e: " & swPartingLineFeatureData.PullDirectionType)
        Debug.Print(" Split faces: " & swPartingLineFeatureData.SplitFaces)
        If swPartingLineFeatureData.SplitFaces Then
            Debug.Print(" Split faces options as defined by swSplitFacesOption_e: " & swPartingLineFeatureData.SplitFacesOption)
        End If
        Debug.Print(" Core/cavity split: " & swPartingLineFeatureData.CoreCavitySplit)
        Debug.Print(" Number of entities to split: " & swPartingLineFeatureData.GetEntitiesToSplitCount)

        'Modify and roll forward parting line feature
        swFeature.ModifyDefinition(swPartingLineFeatureData, swModel, Nothing)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get and Set Parting Surface Feature Data Example (C#)

This example shows how to get and set parting surface feature data.

```
//-----
// Preconditions:
// 1. Open a model document that contains a parting surface feature.
// 2. Select the parting surface feature in the FeatureManager design tree.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Gets and sets parting surface feature data.
// 2. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace PartingSurfaceFeatureDataCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            Feature swFeature = default(Feature);
            PartingSurfaceFeatureData swPartingSurfaceFeatureData = default(PartingSurfaceFeatureData);
            bool state = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;

            //Get parting surface feature
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFeature = (Feature)swSelectionMgr.GetSelectedObject6(1, -1);
            swPartingSurfaceFeatureData = (PartingSurfaceFeatureData)swFeature.GetDefinition();

            //Roll back to get and set parting surface feature data
            state = swPartingSurfaceFeatureData.AccessSelections(swModel, null);
            Debug.Print("File = " + swModel.GetPathName());
            Debug.Print(" Feature name: " + swFeature.Name);
            Debug.Print(" Number of parting lines: " + swPartingSurfaceFeatureData.GetPartingLinesCount());
            Debug.Print(" Parting line type (0 = parting line feature, 1 = edge): " + swPartingSurfaceFeatureData.GetPartingLinesType());
            Debug.Print(" Parting surface type as defined by swPartingSurfaceMoldParmType_e: " + swPartingSurfaceFeatureData.PartingType);
            if (swPartingSurfaceFeatureData.PartingType == (int)swPartingSurfaceMoldParmType_e.swPartingSurfaceMoldParmPerpendicular)
            {
                Debug.Print(" Offset angle not available for this parting surface type.");
            }
            else
            {
                Debug.Print(" Offset angle: " + swPartingSurfaceFeatureData.OffsetAngle);
            }
            Debug.Print(" Knit all surfaces: " + swPartingSurfaceFeatureData.Knit);
            Debug.Print(" Distance this parting surface feature extends: " + swPartingSurfaceFeatureData.OffsetDistance);
            Debug.Print(" Transition between adjacent edges as defined by swPartingSurfaceSmoothingType_e: " + swPartingSurfaceFeatureData.TransitionType);

            //Reverse alignment
            if (swPartingSurfaceFeatureData.ReverseAlignment == true)
            {
                swPartingSurfaceFeatureData.ReverseAlignment = false;
            }
            else
            {
                swPartingSurfaceFeatureData.ReverseAlignment = true;
            }
            Debug.Print(" Reversed alignment: " + swPartingSurfaceFeatureData.ReverseAlignment);

            //Modify and roll forward parting surface feature
            swFeature.ModifyDefinition(swPartingSurfaceFeatureData, swModel, null);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get and Set Parting Surface Feature Data Example (VBA)

This example shows how to get and set parting surface feature data.

```

'-----
' Preconditions:
' 1. Open a model document that contains a parting surface feature.
' 2. Select the parting surface feature in the FeatureManager design tree.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Gets and sets parting surface feature data.
' 2. Examine the Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFeature As SldWorks.Feature
Dim swPartingSurfaceFeatureData As SldWorks.PartingSurfaceFeatureData
Dim state As Boolean

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    'Get parting surface feature
    Set swSelectionMgr = swModel.SelectionManager
    Set swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
    Set swPartingSurfaceFeatureData = swFeature.GetDefinition

    'Roll back to get and set parting surface feature data
    state = swPartingSurfaceFeatureData.AccessSelections(swModel, Nothing)
    Debug.Print "File = " & swModel.GetPathName
    Debug.Print " Feature name: " & swFeature.Name
    Debug.Print " Number of parting lines: " & swPartingSurfaceFeatureData.GetPartingLinesCount
    Debug.Print " Parting line type (0 = parting line feature, 1 = edge): " & swPartingSurfaceFeatureData.GetPartingLinesType
    Debug.Print " Parting surface type as defined by swPartingSurfaceMoldParmType_e: " & swPartingSurfaceFeatureData.PartingType
    If swPartingSurfaceFeatureData.PartingType = swPartingSurfaceMoldParmType_e.swPartingSurfaceMoldParmPerpendicular Then
        Debug.Print " Offset angle not available for this parting surface type."
    Else
        Debug.Print " Offset angle: " & swPartingSurfaceFeatureData.OffsetAngle
    End If
    Debug.Print " Knit all surfaces: " & swPartingSurfaceFeatureData.Knit
    Debug.Print " Distance this parting surface feature extends: " & swPartingSurfaceFeatureData.OffsetDistance
    Debug.Print " Transition between adjacent edges as defined by swPartingSurfaceSmoothingType_e: " & swPartingSurfaceFeatureData.TransitionType
    'Reverse alignment
    If (swPartingSurfaceFeatureData.ReverseAlignment = True) Then
        swPartingSurfaceFeatureData.ReverseAlignment = False
    Else
        swPartingSurfaceFeatureData.ReverseAlignment = True
    End If
    Debug.Print (" Reversed alignment: " & swPartingSurfaceFeatureData.ReverseAlignment)

    'Modify and roll forward parting surface feature
    swFeature.ModifyDefinition swPartingSurfaceFeatureData, swModel, Nothing
End Sub

```

Get and Set Parting Surface Feature Data Example (VB.NET)

This example shows how to get and set parting surface feature data.

```

-----
Preconditions:
1. Open a model document that contains a parting surface feature.
2. Select the parting surface feature in the FeatureManager design tree.
3. Open the Immediate window.
Postconditions:
1. Gets and sets parting surface feature data.
2. Examine the Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelectionMgr As SelectionMgr
        Dim swFeature As Feature
        Dim swPartingSurfaceFeatureData As PartingSurfaceFeatureData
        Dim state As Boolean

        swModel = swApp.ActiveDoc

        'Get parting surface feature
        swSelectionMgr = swModel.SelectionManager
        swFeature = swSelectionMgr.GetSelectedObject6(1, -1)
        swPartingSurfaceFeatureData = swFeature.GetDefinition

        'Roll back to get and set parting surface feature data
        state = swPartingSurfaceFeatureData.AccessSelections(swModel, Nothing)
        Debug.Print("File = " & swModel.GetPathName)
        Debug.Print(" Feature name: " & swFeature.Name)
        Debug.Print(" Number of parting lines: " & swPartingSurfaceFeatureData.GetPartingLinesCount)
        Debug.Print(" Parting line type (0 = parting line feature, 1 = edge): " & swPartingSurfaceFeatureData.GetPartingLinesType)
        Debug.Print(" Parting surface type as defined by swPartingSurfaceMoldParmType_e: " & swPartingSurfaceFeatureData.PartingType)
        If swPartingSurfaceFeatureData.PartingType = swPartingSurfaceMoldParmType_e.swPartingSurfaceMoldParmPerpendicular Then
            Debug.Print(" Offset angle not available for this parting surface type.")
        Else
            Debug.Print(" Offset angle: " & swPartingSurfaceFeatureData.OffsetAngle)
        End If
        Debug.Print(" Knit all surfaces: " & swPartingSurfaceFeatureData.Knit)
        Debug.Print(" Distance this parting surface feature extends: " & swPartingSurfaceFeatureData.OffsetDistance)
        Debug.Print(" Transition between adjacent edges as defined by swPartingSurfaceSmoothingType_e: " & swPartingSurfaceFeatureData.TransitionType)
        'Reverse alignment
        If (swPartingSurfaceFeatureData.ReverseAlignment = True) Then
            swPartingSurfaceFeatureData.ReverseAlignment = False
        Else
            swPartingSurfaceFeatureData.ReverseAlignment = True
        End If
        Debug.Print(" Reversed alignment: " & swPartingSurfaceFeatureData.ReverseAlignment)

        'Modify and roll forward parting surface feature
        swFeature.ModifyDefinition(swPartingSurfaceFeatureData, swModel, Nothing)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get and Set Routing Component Grouping Options for BOM Table Example (C#)

This example shows how to get and set the routing component grouping options for this BOM table in a drawing of an assembly containing routing components.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\AutoRouteThroughSketchEntities.slddrw.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Selects the Bill of Materials1 feature.
// 2. Examine the Immediate window.
//
// NOTE: Because the drawing is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            BomFeature swBomFeature = default(BomFeature);
            bool status = false;
            int options = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Bill of Materials1", "BOMFEATURE", 0, 0, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swBomFeature = (BomFeature)swSelMgr.GetSelectedObject6(1, -1);
            Debug.Print("Name of configuration used for BOM table: " + swBomFeature.Configuration);

            //Get current routing component grouping options
            Debug.Print("Current routing component grouping options: " + swBomFeature.RoutingComponentGrouping);

            //Set new routing component grouping options
            options = (int)swRoutingComponentGroupingOption_e.swShowOnlyRoutingComponentsInBOM + (int)swRoutingComponentGroupingOption_e.swDisplayUnitsInBOM;

            swBomFeature.RoutingComponentGrouping = options;
            Debug.Print("Modified routing component grouping options: " + swBomFeature.RoutingComponentGrouping);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Get and Set Routing Component Grouping Options for BOM Table Example (VBA)

This example shows how to get and set the routing component grouping options for this BOM table in a drawing of an assembly containing routing components.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\AutoRouteThroughSketchEntities.slddrwr.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects the Bill of Materials1 feature.
' 2. Examine the Immediate window.
'
' NOTE: Because the drawing is used elsewhere, do not save changes.
'-----
```

Option Explicit

Sub main()

```
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swBomFeature As SldWorks.BomFeature
    Dim status As Boolean
    Dim options As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    status = swModelDocExt.SelectByID2("Bill of Materials1", "BOMFEATURE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set swBomFeature = swSelMgr.GetSelectedObject6(1, -1)
    Debug.Print "Name of configuration used for BOM table: " & swBomFeature.Configuration

    'Get current routing component grouping options
    Debug.Print "Current routing component grouping options: " & swBomFeature.RoutingComponentGrouping

    'Set new routing component grouping options
    options = swRoutingComponentGroupingOption_e.swShowOnlyRoutingComponentsInBOM + swRoutingComponentGroupingOption_e.swDisplayUnitsInBOM
    swBomFeature.RoutingComponentGrouping = options
    Debug.Print "Modified routing component grouping options: " & swBomFeature.RoutingComponentGrouping
```

End Sub

Get and Set Routing Component Grouping Options for BOM Table Example (VB.NET)

This example shows how to get and set the routing component grouping options for this BOM table in a drawing of an assembly containing routing components.

```

-----
Preconditions:
1. Open public_documents\samples\tutorial\api\AutoRouteThroughSketchEntities.slddrw.
2. Open the Immediate window.
Postconditions:
1. Selects the Bill of Materials1 feature.
2. Examine the Immediate window.
NOTE: Because the drawing is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swBomFeature As BomFeature
        Dim status As Boolean
        Dim options As Integer

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Bill of Materials1", "BOMFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        swBomFeature = swSelMgr.GetSelectedObject6(1, -1)
        Debug.Print("Name of configuration used for BOM table: " & swBomFeature.Configuration)

        'Get current routing component grouping options
        Debug.Print("Current routing component grouping options: " & swBomFeature.RoutingComponentGrouping)

        'Set new routing component grouping options
        options = swRoutingComponentGroupingOption_e.swShowOnlyRoutingComponentsInBOM + swRoutingComponentGroupingOption_e.swDisplayUnitsInBOM
        swBomFeature.RoutingComponentGrouping = options
        Debug.Print("Modified routing component grouping options: " & swBomFeature.RoutingComponentGrouping)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Get and Set Scene Properties Example (C#)

This example shows how to get a model's scene and get and set some scene properties.

```
//-----
// Preconditions:
// 1. Open a model document.
// 2. Open an Immediate window.
//
// Postconditions: Inspect the Immediate window.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace Scene_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        SWScene Scene;
        ModelDoc2 swModel;
        Configuration swConfig;
        MathPoint swPoint;
        MathVector swVector;
        double[] point;
        double[] vect;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;

            swConfig = (Configuration)swModel.GetActiveConfiguration();
            Debug.Print("Configuration: " + swConfig.Name);

            Scene = swConfig.GetScene();

            string P2SFilename = null;
            Scene.GetP2SFileName(ref P2SFilename);
            Debug.Print("Scene file: " + P2SFilename);

            Debug.Print("Scene background top gradient color: " + Scene.BackgroundTopGradientColor);
            Debug.Print("Scene background bottom gradient color: " + Scene.BackgroundBottomGradientColor);

            Scene.GetFloorNormal(ref swPoint, ref swVector);
            point = (double[])swPoint.ArrayData;
            Debug.Print("Scene floor normal point: " + point[0] + ", " + point[1] + ", " + point[2]);
            vect = (double[])swVector.ArrayData;
            Debug.Print("Scene floor normal vector: " + vect[0] + ", " + vect[1] + ", " + vect[2]);

            Scene.BackgroundColor = (int)swSceneBackgroundColor_e.swBackgroundColor_Environment;
            Debug.Print("Type of scene background as defined in swSceneBackgroundColor_e: " + Scene.BackgroundColor);
            Debug.Print("Scene background environment image file: " + Scene.BackgroundImage);
            Debug.Print("Scene background image file: " + Scene.BackgroundImage);
            Debug.Print("Scene environment rotation: " + Scene.EnvironmentRotation);
            Scene.FitToSWindow = true;
            Debug.Print("Stretch to fit in SOLIDWORKS Design window? " + Scene.FitToSWindow);
            Debug.Print("Scale the scene floor uniformly? " + Scene.FixedAspectRatio);
            Debug.Print("Flip the scene floor direction? " + Scene.FloorDirection);
            Debug.Print("Automatically resize the scene floor based on the model bounding box? " + Scene.FloorAutoSize);
            Debug.Print("Distance between scene floor and model: " + Scene.FloorOffset);
            Debug.Print("Flip the scene floor offset direction? " + Scene.FloorOffsetDirection);
            Scene.FloorReflections = true;
            Debug.Print("Show model reflections on the scene floor? " + Scene.FloorReflections);
            Debug.Print("Scene floor rotation: " + Scene.FloorRotation);
            Debug.Print("Show model shadows on the scene floor? " + Scene.FloorShadows);
            Debug.Print("Keep the scene background when changing the scene? " + Scene.KeepBackground);
            Scene.FlattenFloor = true;
            Debug.Print("Flatten the scene floor of the spherical environment? " + Scene.FlattenFloor);
            Debug.Print("Horizon height: " + Scene.HorizonHeight);
            Debug.Print("Environment size: " + Scene.EnvironmentSize);

        }

        public SldWorks swApp;
    }
}
```

Get and Set Scene Properties Example (VBA)

This example shows how to get a model's scene and get and set some scene properties.

```

'-----
' Preconditions:
' 1. Open a model document.
' 2. Open an Immediate window.
'
' Postconditions: Inspect the Immediate window.
'-----

Option Explicit
Dim Scene As SldWorks.swScene
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swConfig As SldWorks.Configuration
Dim swPoint As SldWorks.MathPoint
Dim swVector As SldWorks.MathVector
Dim point As Variant
Dim vect As Variant

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Set swConfig = swModel.GetActiveConfiguration
    Debug.Print "Configuration: " & swConfig.name

    Set Scene = swConfig.GetScene

    Dim P2SFilename As String
    Scene.GetP2SFileName P2SFilename
    Debug.Print "Scene file: " & P2SFilename

    Debug.Print "Scene background top gradient color: " & Scene.BackgroundTopGradientColor
    Debug.Print "Scene background bottom gradient color: " & Scene.BackgroundBottomGradientColor

    Scene.GetFloorNormal swPoint, swVector
    point = swPoint.ArrayData
    Debug.Print "Scene floor normal point: " & point(0) & ", " & point(1) & ", " & point(2)
    vect = swVector.ArrayData
    Debug.Print "Scene floor normal vector: " & vect(0) & ", " & vect(1) & ", " & vect(2)

    Scene.BackgroundType = swSceneBackgroundType_e.swBackgroundType_UseEnvironment
    Debug.Print "Type of scene background as defined in swSceneBackgroundType_e: " & Scene.BackgroundType
    Debug.Print "Scene background environment image file: " & Scene.BackgroundEnvImage
    Debug.Print "Scene background image file: " & Scene.BackgroundImage
    Debug.Print "Scene environment rotation: " & Scene.EnvironmentRotation
    Scene.FitToSWWindow = True
    Debug.Print "Stretch to fit in SOLIDWORKS Design window? " & Scene.FitToSWWindow
    Debug.Print "Scale the scene floor uniformly? " & Scene.FixedAspectRatio
    Debug.Print "Flip the scene floor direction? " & Scene.FloorDirection
    Debug.Print "Automatically resize the scene floor based on the model bounding box? " & Scene.FloorAutoSize
    Debug.Print "Distance between scene floor and model: " & Scene.FloorOffset
    Debug.Print "Flip the scene floor offset direction? " & Scene.FloorOffsetDirection
    Scene.FloorReflections = True
    Debug.Print "Show model reflections on the scene floor? " & Scene.FloorReflections
    Debug.Print "Scene floor rotation: " & Scene.FloorRotation
    Debug.Print "Show model shadows on the scene floor? " & Scene.FloorShadows
    Debug.Print "Keep the scene background when changing the scene? " & Scene.KeepBackground
    Scene.FlattenFloor = True
    Debug.Print "Flatten the scene floor of the spherical environment? " & Scene.FlattenFloor
    Debug.Print "Horizon height: " & Scene.HorizonHeight
    Debug.Print "Environment size: " & Scene.EnvironmentSize
End Sub

```

Get and Set Scene Properties Example (VB.NET)

This example shows how to get a model's scene and get and set some scene properties.

```

'-----
' Preconditions:
' 1. Open a model document.
' 2. Open an Immediate window.
'-----
' Postconditions: Inspect the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Scene As SWScene
    Dim swModel As ModelDoc2
    Dim swConfig As Configuration
    Dim swPoint As MathPoint
    Dim swVector As MathVector
    Dim point As Object
    Dim vect As Object

    Sub main()

        swModel = swApp.ActiveDoc

        swConfig = swModel.GetActiveConfiguration
        Debug.Print("Configuration: " & swConfig.name)

        Scene = swConfig.GetScene

        Dim P2SFilename As String = Nothing
        Scene.GetP2SFileName(P2SFilename)
        Debug.Print("Scene file: " & P2SFilename)

        Debug.Print("Scene background top gradient color: " & Scene.BackgroundTopGradientColor)
        Debug.Print("Scene background bottom gradient color: " & Scene.BackgroundBottomGradientColor)

        Scene.GetFloorNormal(swPoint, swVector)
        point = swPoint.ArrayData
        Debug.Print("Scene floor normal point: " & point(0) & ", " & point(1) & ", " & point(2))
        vect = swVector.ArrayData
        Debug.Print("Scene floor normal vector: " & vect(0) & ", " & vect(1) & ", " & vect(2))

        Scene.BackgroundColor = swSceneBackgroundColor_e.swBackgroundColor_Environment
        Debug.Print("Type of scene background as defined in swSceneBackgroundColor_e: " & Scene.BackgroundColor)
        Debug.Print("Scene background environment image file: " & Scene.BackgroundImageEnvImage)
        Debug.Print("Scene background image file: " & Scene.BackgroundImage)
        Debug.Print("Scene environment rotation: " & Scene.EnvironmentRotation)
        Scene.FitToSWWindow = True
        Debug.Print("Stretch to fit in SOLIDWORKS Design window? " & Scene.FitToSWWindow)
        Debug.Print("Scale the scene floor uniformly? " & Scene.FixedAspectRatio)
        Debug.Print("Flip the scene floor direction? " & Scene.FloorDirection)
        Debug.Print("Automatically resize the scene floor based on the model bounding box? " & Scene.FloorAutoSize)
        Debug.Print("Distance between scene floor and model: " & Scene.FloorOffset)
        Debug.Print("Flip the scene floor offset direction? " & Scene.FloorOffsetDirection)
        Scene.FloorReflections = True
        Debug.Print("Show model reflections on the scene floor? " & Scene.FloorReflections)
        Debug.Print("Scene floor rotation: " & Scene.FloorRotation)
        Debug.Print("Show model shadows on the scene floor? " & Scene.FloorShadows)
        Debug.Print("Keep the scene background when changing the scene? " & Scene.KeepBackground)
        Scene.FlattenFloor = True
        Debug.Print("Flatten the scene floor of the spherical environment? " & Scene.FlattenFloor)
        Debug.Print("Horizon height: " & Scene.HorizonHeight)
        Debug.Print("Environment size: " & Scene.EnvironmentSize)

    End Sub

    Public swApp As SldWorks

End Class

```

Get and Set Search Folders Example (C#)

This example shows how to get and set the file locations in which SOLIDWORKS Design searches for reference documents.

```
//-----
// Preconditions: Open the Immediate window.
//
// Postconditions:
// 1. Adds c:\; to the beginning of the search folder path for reference
// documents.
// 2. Examine the Immediate window.
//
// NOTE: This macro changes your SOLIDWORKS Design search folders system settings.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        string searchFolders;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swUseFolderSearchRules, false);

            searchFolders = swApp.GetSearchFolders((int)swSearchFolderTypes_e.swDocumentType);
            Debug.Print("Current search path: " + searchFolders);
            searchFolders = "c:\\;" + searchFolders;
            boolstatus = swApp.SetSearchFolders((int)swSearchFolderTypes_e.swDocumentType, searchFolders);
            searchFolders = swApp.GetSearchFolders((int)swSearchFolderTypes_e.swDocumentType);
            Debug.Print("New search path: " + searchFolders);
        }

        public SldWorks swApp;
    }
}
```

Get and Set Search Folders Example (VBA)

This example shows how to get and set the file locations in which SOLIDWORKS Design searches for reference documents.

```
'-----  
' Preconditions: Open the Immediate window.  
'  
' Postconditions:  
' 1. Adds c:\; to the beginning of the search folder path for reference  
'   documents.  
' 2. Inspect the Immediate window.  
'  
' NOTE: This macro changes your SOLIDWORKS Design search folders system settings.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim searchFolders As String  
Dim boolstatus As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
  
    swApp.SetUserPreferenceToggle swUserPreferenceToggle_e.swUseFolderSearchRules, False  
  
    searchFolders = swApp.GetSearchFolders(swSearchFolderTypes_e.swDocumentType)  
    Debug.Print "Current search path: " & searchFolders  
    searchFolders = "c:\;" & searchFolders  
    boolstatus = swApp.SetSearchFolders(swSearchFolderTypes_e.swDocumentType, searchFolders)  
    searchFolders = swApp.GetSearchFolders(swSearchFolderTypes_e.swDocumentType)  
    Debug.Print "New search path: " & searchFolders  
  
End Sub
```

Get and Set Search Folders Example (VB.NET)

This example shows how to get and set the file locations in which SOLIDWORKS Design searches for reference documents.

```
'-----  
' Preconditions: Open the Immediate window.  
'  
' Postconditions:  
' 1. Adds c:\; to the beginning of the search folder path for reference  
'   documents.  
' 2. Inspect the Immediate window.  
' NOTE: This macro changes your SOLIDWORKS Design search folders system settings.  
'-----  
  
Imports SolidWorks.Interop.sldworks  
  
Imports SolidWorks.Interop.swconst  
  
Imports System.Runtime.InteropServices  
  
Imports System  
  
Imports System.Diagnostics  
  
  
Partial Class SolidWorksMacro  
  
    Dim Part As ModelDoc2  
  
    Dim searchFolders As String  
  
    Dim boolstatus As Boolean  
  
    Sub main()  
  
        Part = swApp.ActiveDoc  
  
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swUseFolderSearchRules, False)  
  
        searchFolders = swApp.GetSearchFolders(swSearchFolderTypes_e.swDocumentType)  
        Debug.Print "Current search path: " & searchFolders  
        searchFolders = "c:\;" & searchFolders  
        boolstatus = swApp.SetSearchFolders(swSearchFolderTypes_e.swDocumentType, searchFolders)  
        searchFolders = swApp.GetSearchFolders(swSearchFolderTypes_e.swDocumentType)  
        Debug.Print "New search path: " & searchFolders  
  
    End Sub  
  
  
    Public swApp As SldWorks  
  
End Class
```

Get and Set Seed Components Example (C#)

This example shows how to access the seed components of patterns in an assembly. It also shows that seed components can be replaced by either IComponent2 objects or IFeature objects representing those components.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly exists.
// 2. Open the Immediate Window.
//
// Postconditions:
// 1. Gets the names of the seed components.
// 2. Replaces the seed components either by an IComponent2 object
//    or an IFeature object representing the component.
// 3. Examine the Immediate window.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace AssemblyPatterns.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            Dispatcher[] dispWrapArr = null;

            string fileName = null;
            int errors = 0;
            int warnings = 0;
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\assem2.sldasm";

            // Open the model
            ModelDoc2 swModel = default(ModelDoc2);
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY,
            (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            // Verify that the model is active
            if (swModel == null)
            {
                Debug.Print("No active model document");
                return;
            }

            Debug.Print("Model: " + swModel.GetTitle());

            // Get all of the features in the model
            FeatureManager swFeatMgr = default(FeatureManager);
            swFeatMgr = (FeatureManager)swModel.FeatureManager;

            object[] vFeatures = null;
            vFeatures = (object[])swFeatMgr.GetFeatures(true);

            // Iterate over all features
            for (int iFeat = 0; iFeat <= vFeatures.GetUpperBound(0); iFeat++)
            {
                Feature swFeature = default(Feature);
                swFeature = (Feature)vFeatures[iFeat];

                // Is the current feature a patterned feature?
                string name = "";
                name = swFeature.GetTypeName2();
                name = name.ToUpper();

                switch (name)
                {
                    case "LOCALLPATTERN":
                        // ILocalLinearPatternFeatureData
                        Debug.Print("    Linear Pattern: " + swFeature.Name);
                        LocalLinearPatternFeatureData llpDefinition = default(LocalLinearPatternFeatureData);
                        llpDefinition = (LocalLinearPatternFeatureData)swFeature.GetDefinition();
                        object[] llpSeedComps = null;

                        if (llpDefinition.AccessSelections(swModel, null))
                        {
                            // Get the seed components for this pattern
                            llpSeedComps = (object[])llpDefinition.SeedComponentArray;
                            Array.Resize(ref llpSeedComps, (llpSeedComps.GetUpperBound(0) + 1));
                        }

                        // Arbitrarily decide whether to replace each seed component
                        // with a component or a feature
                        ProcessSeedComponentArray(ref llpSeedComps);
                    }
                }
            }
        }
    }
}
```

```

        dispWrapArr = (DispatchWrapper[])ObjectArrayToDispatchWrapperArray(llpSeedComps);
        llpDefinition.SeedComponentArray = dispWrapArr;

        // Update the feature definition
        swFeature.ModifyDefinition(llpDefinition, swModel, null);

        break;
    case "LOCALCIRPATTERN":
        // ILocalCircularPatternFeatureData
        Debug.Print("    Circular Pattern: " + swFeature.Name);
        LocalCircularPatternFeatureData cpDefinition = default(LocalCircularPatternFeatureData);
        cpDefinition = (LocalCircularPatternFeatureData)swFeature.GetDefinition();
        object[] cpSeedComps = null;

        if (cpDefinition.AccessSelections(swModel, null))
        {
            // Get the seed components for this pattern
            cpSeedComps = (object[])cpDefinition.SeedComponentArray;
            Array.Resize(ref cpSeedComps, (cpSeedComps.GetUpperBound(0) + 1));
        }

        // Arbitrarily decide whether to replace each seed component
        // with a component or a feature
        ProcessSeedComponentArray(ref cpSeedComps);

        dispWrapArr = (DispatchWrapper[])ObjectArrayToDispatchWrapperArray(cpSeedComps);
        cpDefinition.SeedComponentArray = dispWrapArr;

        // Update the feature definition
        swFeature.ModifyDefinition(cpDefinition, swModel, null);

        break;
    case "DERIVEDLPATTERN":
        // IDerivedPatternFeatureData
        Debug.Print("    Derived Linear Pattern: " + swFeature.Name);
        DerivedPatternFeatureData dpDefinition = default(DerivedPatternFeatureData);
        dpDefinition = (DerivedPatternFeatureData)swFeature.GetDefinition();
        object[] dpSeedComps = null;
        if (dpDefinition.AccessSelections(swModel, null))
        {
            // Get the seed components for this pattern
            dpSeedComps = (object[])dpDefinition.SeedComponentArray;
            Array.Resize(ref dpSeedComps, (dpSeedComps.GetUpperBound(0) + 1));
        }

        // Arbitrarily decide whether to replace each seed component
        // with a component or a feature
        ProcessSeedComponentArray(ref dpSeedComps);

        dispWrapArr = (DispatchWrapper[])ObjectArrayToDispatchWrapperArray(dpSeedComps);
        dpDefinition.SeedComponentArray = dispWrapArr;

        // Update the feature definition
        swFeature.ModifyDefinition(dpDefinition, swModel, null);

        break;
    case "DERIVEDCIRPATTERN":
        // IDerivedPatternFeatureData
        Debug.Print("    Derived Circular Pattern: " + swFeature.Name);
        DerivedPatternFeatureData dcpDefinition = default(DerivedPatternFeatureData);
        dcpDefinition = (DerivedPatternFeatureData)swFeature.GetDefinition();
        object[] dcpSeedComps = null;
        if (dcpDefinition.AccessSelections(swModel, null))
        {
            // Get the seed components for this pattern
            dcpSeedComps = (object[])dcpDefinition.SeedComponentArray;
            Array.Resize(ref dcpSeedComps, (dcpSeedComps.GetUpperBound(0) + 1));
        }

        // Arbitrarily decide whether to replace each seed component
        // with a component or a feature
        ProcessSeedComponentArray(ref dcpSeedComps);

        dispWrapArr = (DispatchWrapper[])ObjectArrayToDispatchWrapperArray(dcpSeedComps);
        dcpDefinition.SeedComponentArray = dispWrapArr;

        // Update the feature definition
        swFeature.ModifyDefinition(dcpDefinition, swModel, null);

        break;
    }
}

private void ProcessSeedComponentArray(ref object[] vSeedComps)
{
    object[] replacementSeeds = null;
    Array.Resize(ref replacementSeeds, (vSeedComps.GetUpperBound(0) + 1));

    // Iterate over each seed component
    int iSeed = 0;
    for (iSeed = 0; iSeed <= vSeedComps.GetUpperBound(0); iSeed++)
    {
        Feature swCompFeat = default(Feature);
        swCompFeat = (Feature)vSeedComps[iSeed];
    }
}

```

```

        Debug.Print("          Seed " + iSeed + ": " + swCompFeat.Name);

        // Access the seed component represented by the feature
        Component2 swComp = default(Component2);
        swComp = (Component2)swCompFeat.GetSpecificFeature2();

        // Arbitrarily decide whether to replace this seed component
        // with a component or a feature
        if (iSeed % 2 == 0)
        {
            replacementSeeds[iSeed] = swComp;
        }
        else
        {
            replacementSeeds[iSeed] = swCompFeat;
        }
    }

    // Replace the seed array
    vSeedComps = (object[])replacementSeeds;
}

public Dispatcher[] ObjectArrayToDispatcherArray(object[] Objects)
{
    int ArraySize = 0;
    ArraySize = Objects.GetUpperBound(0);
    Dispatcher[] d = new Dispatcher[ArraySize + 1];
    int ArrayIndex = 0;
    for (ArrayIndex = 0; ArrayIndex <= ArraySize; ArrayIndex++)
    {
        d[ArrayIndex] = new Dispatcher(Objects[ArrayIndex]);
    }
    return d;
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Get and Set Seed Components Example (VBA)

This example shows how to access the seed components of patterns in an assembly. It also shows that seed components can be replaced either by IComponent2 objects or IFeature objects representing those components.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Open the Immediate Window.
'
' Postconditions:
' 1. Gets the names of the seed components.
' 2. Replaces the seed components either by an IComponent2 object
'    or an IFeature object representing the component.
' 3. Examine the Immediate window.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----

```

Option Explicit

```

Sub main()
    ' Attach to SOLIDWORKS Design
    Dim swApp As SldWorks.SldWorks
    Set swApp = Application.SldWorks

    Dim fileName As String
    Dim errors As Long
    Dim warnings As Long
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\assem2.sldasm"

    ' Open the model
    Dim swModel As ModelDoc2
    Set swModel = swApp.OpenDoc6(fileName, swDocASSEMBLY, swOpenDocOptions_Silent, "", errors, warnings)

    ' Verify that the model is active
    If swModel Is Nothing Then
        Debug.Print "No active model document"
        Exit Sub
    End If

    Debug.Print "Model: " & swModel.GetTitle

    ' Get all of the features in the model
    Dim swFeatMgr As FeatureManager
    Set swFeatMgr = swModel.FeatureManager

    Dim vFeatures As Variant
    vFeatures = swFeatMgr.GetFeatures(True)

    ' Iterate over all features
    Dim iFeat As Integer
    For iFeat = LBound(vFeatures) To UBound(vFeatures)
        Dim swFeature As Feature
        Set swFeature = vFeatures(iFeat)

        ' Is the current feature a patterned feature?
        Select Case UCase(swFeature.GetTypeName2)
            Case "LOCALLPATTERN"
                ' ILocalLinearPatternFeatureData
                Debug.Print "    Linear Pattern: " & swFeature.Name
                ProcessGenericPatternDefinition swFeature, swModel

            Case "LOCALCIRPATTERN"
                ' ILocalCircularPatternFeatureData
                Debug.Print "    Circular Pattern: " & swFeature.Name
                ProcessGenericPatternDefinition swFeature, swModel

            Case "DERIVEDLPATTERN"
                ' IDerivedPatternFeatureData
                Debug.Print "    Derived Linear Pattern: " & swFeature.Name
                ProcessGenericPatternDefinition swFeature, swModel

            Case "DERIVEDCIRPATTERN"
                ' IDerivedPatternFeatureData
                Debug.Print "    Derived Circular Pattern: " & swFeature.Name
                ProcessGenericPatternDefinition swFeature, swModel

        End Select
    Next

End Sub

Private Sub ProcessGenericPatternDefinition(swFeature As Feature, swModel As ModelDoc2)

    Dim patternDef As Object
    Set patternDef = swFeature.GetDefinition

    ' Verify the type of feature data object
    If TypeOf patternDef Is DerivedPatternFeatureData Or _
        TypeOf patternDef Is LocalCircularPatternFeatureData Or _
        TypeOf patternDef Is LocalLinearPatternFeatureData Then

        If patternDef.AccessSelections(swModel, Nothing) Then
            ' Get the seed components for this pattern
            Dim vSeedComps As Variant
            vSeedComps = patternDef.SeedComponentArray

            ProcessSeedComponentArray vSeedComps
        End If
    End If
End Sub

```

```

        ' Replace the seed array
        patternDef.SeedComponentArray = vSeedComps

        ' Update the feature definition
        swFeature.ModifyDefinition patternDef, swModel, Nothing
    End If
End Sub

Private Sub ProcessSeedComponentArray(ByRef vSeedComps As Variant)
    ' Replace the current seeds with themselves as either features or components
    Dim replacementSeeds() As Object
    ReDim replacementSeeds(LBound(vSeedComps) To UBound(vSeedComps))

    ' Iterate over each seed component
    Dim iSeed As Integer
    For iSeed = LBound(vSeedComps) To UBound(vSeedComps)
        Dim swCompFeat As Feature
        Set swCompFeat = vSeedComps(iSeed)

        Debug.Print "        Seed " & iSeed & ": " & swCompFeat.Name

        ' Access the seed component represented by the feature
        Dim swComp As Component2
        Set swComp = swCompFeat.GetSpecificFeature2

        ' Arbitrarily decide whether to replace this seed component
        ' with a component or a feature
        If iSeed Mod 2 = 0 Then
            Set replacementSeeds(iSeed) = swComp
        Else
            Set replacementSeeds(iSeed) = swCompFeat
        End If
    Next

    ' Replace the seed array
    vSeedComps = replacementSeeds
End Sub

```